

Chapter 7

Design Patterns and Software Refactoring

7.1 Chapter Objectives

1. Be able to know what design patterns are.
2. From a given list, select the most appropriate pattern for a given scenario and demonstrate its applicability
3. From a list, select the most appropriate pattern for a given scenario. Patterns are limited to those documented in the book - Gamma, Erich; Richard Helm, Ralph Johnson, and John Vlissides (1995). Design Patterns: Elements of Reusable Object-Oriented Software and are named using the names given in that book.
4. Understand the idea of refactoring and re-factor software code to Design Patterns

7.2 What are Patterns

Christopher Alexander talking about buildings and towns *"Each pattern describes a problem which occurs over and over again in our environment, and then describes the core of the solution to that problem, in such a way that you can use this solution a million times over, without ever doing it the same way twice"* Alexander, et al., A Pattern Language. Oxford University Press, 1977

Patterns can have different levels of abstraction. In Design Patterns (the book), Patterns are not classes, Patterns are not frameworks. Instead, Patterns are descriptions of communicating objects and classes that are customized to solve a general design problem in a particular context

So, patterns are formalized solutions to design problems. They describe techniques for maximizing flexibility, extensibility, abstraction, etc. These solutions can typically be translated to code in a straight forward manner.

It is important to note the fact that Patterns makes our life easy by giving guidelines to apply in a given context and for a specific problem.

7.3 Elements of Patterns

Pattern Name

This is more than just a handle for referring to the pattern. Each name adds to a designers vocabulary. It enables the discussion of design at a higher abstraction.

The Problem

This gives a detailed description of the problem addressed by the pattern. It describes when to apply a pattern, often with a list of preconditions.

The Solution This describes the elements that make up the design, their relationships, responsibilities, and collaborations. It does not describe a concrete solution. Instead a template to be applied in many situations

The consequences

This describes the results and tradeoffs of applying the pattern. It is critical for evaluating design alternatives Typically include impact on flexibility, extensibility, or portability, Space and Time tradeoffs, Language and Implementation issues

7.4 Design Patterns Templates

1. Pattern Name and Classification
 - Creational
 - Structural
 - Behavioural
2. Intent Also Known As Motivation and Applicability
3. Structure and Participants
4. Collaborations
5. Consequences
6. Implementation
7. Sample Code
8. Known Uses
9. Related Patterns

7.5 Documenting Design Patterns Selected Example Using a Singleton

The selected pattern is Singleton because of its simplicity to grasp and understand.

7.5.1 Singleton

Intent

Ensure a class has only one instance, and provide a global point of access to it

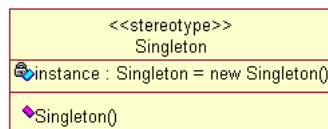
Motivation

Some classes represent objects where multiple instances do not make sense or can lead to a security risk (e.g. Java security managers) and memory or heap overload

Applicability

Use the Singleton pattern when there must be exactly one instance of a class, and it must be accessible to clients from a well-known access point, when the sole instance should be extensible by subclassing, and clients should be able to use an extended instance without modifying their code

Structure



Participants

Just the Singleton class

Collaborations

Clients access a Singleton instance solely through Singleton's Instance operation

Consequences

Controlled access to sole instance. Reduced name space (versus global variables) Permits a variable number of instances (if desired)

Implementation

```
import java.util.Date;

public class Singleton {
    private static Singleton theOnlyOne;
    private Date d = new Date();

    private Singleton() {
    }

    public synchronized static Singleton instance() {
        if (theOnlyOne == null) {
            theOnlyOne = new Singleton();
        }
        return theOnlyOne;
    }

    public Date getDate() {
        return d;
    }
}
```

LISTING 7.1: Singleton Implementation

Usage

```
public class UseSingleton {  
    public static void main(String[] args) {  
        Singleton a = Singleton.getInstance();  
        Singleton b = Singleton.getInstance();  
        System.out.println(" " + a.getDate());  
        System.out.println(" " + b.getDate());  
        System.out.println(" " + a);  
        System.out.println(" " + b);  
    }  
}
```

LISTING 7.2: Singleton Usage

Output:

```
Sun Apr 17 13:03:34 MDT 2011  
Sun Apr 17 13:03:34 MDT 2011  
Singleton@136646  
Singleton@136646
```

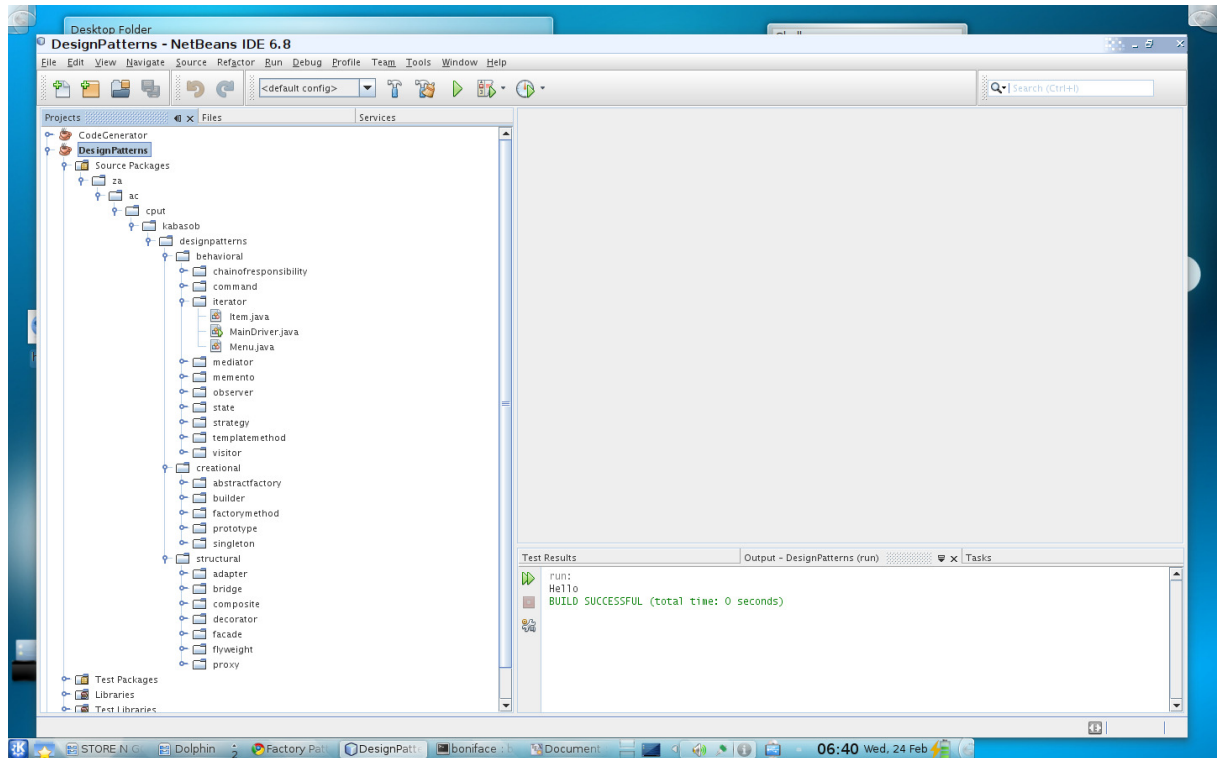
7.6 GOF patterns and Their Classification

There are totally 23 design patterns in GoF (Gang of Four). All these 23 design patterns are mapped to 3 main categories:

1. Creational patterns : Makes Object creation/instantiation job easy.
2. Structural patterns : Makes Objects available to other class by changing the interfaces or contract.
3. Behavioral patterns: Deals with object state changes and it's interaction with other classes/objects.

7.7 GOF Design Patterns Examples in Java

The following section has exercise for the 23 GOF patterns. Create a java application and folders as shown in the diagram below.



And then do the exercises below. Record your output in the log book or word document. zip your application together with your output and upload to WEBCT

7.7.1 Creational Design Patterns

The basic form of object creation could result in design problems or added complexity to the design. Creational design patterns solve this problem by somehow controlling this object creation.

Creational patterns involve the construction of new objects by often hiding the constructors in the classes being created, and provides alternate methods to return instances of the desired class.

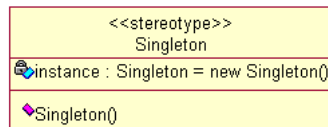
All the creational patterns define the best possible way in which an object can be created considering reuse and changeability. These describes the best way to handle instantiation.

Hard coding the actual instantiation is a pitfall and should be avoided if reuse and changeability are desired. In such scenarios, we can make use of creational patterns to give this a more general and flexible approach.

1. Singleton

2. Factory Method
3. Abstract Factory
4. Builder Pattern
5. Prototype

7.7.1.1 Singleton Pattern



This is another version of the simplest Design Pattern you will ever come across. Essentially, a singleton is a class that is instantiated only once.

This is typically accomplished by creating a static field in the class representing the class. A static method exists on the class to obtain the instance of the class and is typically named something such as `getInstance()`. The creation of the object referenced by the static field can be done either when the class is initialized or the first time that `getInstance()` is called. The singleton class typically has a private constructor to prevent the singleton class from being instantiated via a constructor. Rather, the instance of the singleton is obtained via the static `getInstance()` method.

The `SingletonExample` class is an example of a typical singleton class. It contains a private static `SingletonExample` field. It has a private constructor so that the class can't be instantiated by outside classes. It has a public static `getInstance()` method that returns the one and only `SingletonExample` instance. If this instance doesn't already exist, the `getInstance()` method creates it. The `SingletonExample` class has a public `sayHello()` method that can be used to test the singleton.

```

/*
 * To change this template, choose Tools | Templates
 * and open the template in the editor.
 */
package za.ac.cput.kabasob.designpatterns.creational.singleton;

/**
 *
 * @author boniface
 */
public class SingletonExample {

    private static SingletonExample singletonExample = null;

    private SingletonExample() {

    }

    public static SingletonExample getInstance() {
        if (singletonExample == null) {
            singletonExample = new SingletonExample();
        }
        return singletonExample;
    }

    public void sayHello() {
        System.out.println("Hello");
    }
}
  
```

The `MainDriver` class obtains a `SingletonExample` singleton class via the call to the static `SingletonExample.getInstance()`. We call the `sayHello()` method on the singleton class.

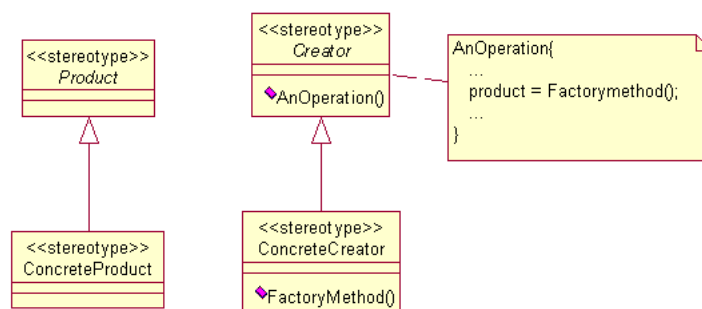
```
/*
 * To change this template, choose Tools | Templates
 * and open the template in the editor.
 */
package za.ac.cput.kabasob.designpatterns.creational.singleton;

/**
 *
 * @author boniface
 */
public class MainDriver {

    public static void main(String[] args) {
        SingletonExample singletonExample = SingletonExample.getInstance();
        singletonExample.sayHello();
    }
}
```

Record output in your log.

7.7.1.2 Factory Method Pattern



The **Factory Pattern** (also known as the **Factory Method Pattern**) is a creational design pattern. A factory is a Java class that is used to encapsulate object creation code. A factory class instantiates and returns a particular type of object based on data passed to the factory. The different types of objects that are returned from a factory typically are subclasses of a common parent class.

The data passed from the calling code to the factory can be passed either when the factory is created or when the method on the factory is called to create an object. This creational method is often called something such as `getInstance` or `getClass`.

As a simple example, let's create an `AnimalFactory` class that will return an animal object based on some data input. To start, here is an abstract `Animal` class. The factory will return an instantiated subclass of `Animal`. `Animal` has a single abstract method, `makeSound()`.

```
package za.ac.cput.kabasob.designpatterns.creational.factorymethod;

/**
 *
 * @author boniface
 */
public abstract class Animal {
    public abstract String makeSound();
}
```

```
}
```

The Dog class is a subclass of Animal. It implements makeSound() to return "Woof".

```
package za.ac.cput.kabasob.designpatterns.creational.factorymethod;
/**
 *
 * @author boniface
 */
public class Dog extends Animal {
    @Override
    public String makeSound() {
        return "Woof";
    }
}
```

The Cat class is a subclass of Animal. It implements makeSound() to return "Meow".

```
package za.ac.cput.kabasob.designpatterns.creational.factorymethod;
/**
 *
 * @author boniface
 */
public class Cat extends Animal {
    @Override
    public String makeSound() {
        return "Meow";
    }
}
```

Now, let's implement our factory. We will call our factory's object creation method getAnimal. This method takes a String as a parameter. If the String is "canine", it returns a Dog object. Otherwise, it returns a Cat object.

```
package za.ac.cput.kabasob.designpatterns.creational.factorymethod;
/**
 *
 * @author boniface
 */
public class AnimalFactory {
    public Animal getAnimal(String type) {
        if ("canine".equals(type)) {
            return new Dog();
        } else {
            return new Cat();
        }
    }
}
```

The MainDriver class demonstrates the use of our factory. It creates an AnimalFactory factory. The factory creates an Animal object and then another Animal object. The first object is a Cat and the second object is a Dog. The output of each object's makeSound() method is displayed.

```
package za.ac.cput.kabasob.designpatterns.creational.factorymethod;
/**
 *
 * @author boniface
 */
public class MainDriver {
    public static void main(String[] args) {
        AnimalFactory animalFactory = new AnimalFactory();

        Animal a1 = animalFactory.getAnimal("feline");
        System.out.println("a1 sound: " + a1.makeSound());

        Animal a2 = animalFactory.getAnimal("canine");
        System.out.println("a2 sound: " + a2.makeSound());
    }
}
```



```
}
}
```

Record your output in your log book.

Notice that the factory has encapsulated our Animal object creation code, thus resulting in clean code in Demo, the class that creates the factory. Additionally, notice the use of polymorphism. We obtain different Animal objects (Cat and Dog) based on data passed to the factory.

Note that it is common to pass data that determines the type of object to be created to the factory when the factory is created (via the factory constructor). However, if multiple objects are being created by the factory, it may make sense to pass this data to the factory's creational method rather than to the constructor, since it might not make sense to create a new factory object each time we wanted to have the factory instantiate a new object.

A factory may also be used in conjunction with the singleton pattern. It is common to have a singleton return a factory instance. To do this, we could replace:

```
AnimalFactory animalFactory = new AnimalFactory();
```

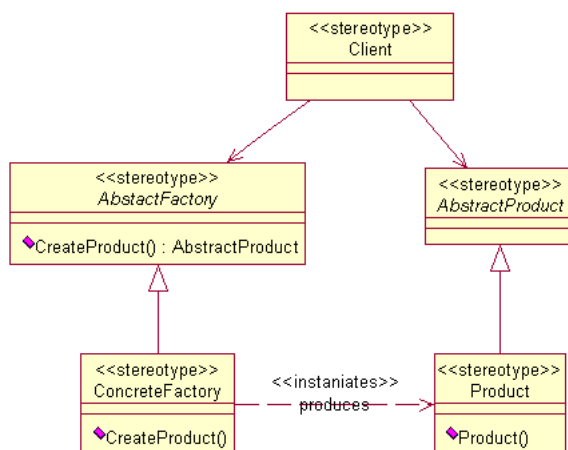
with

```
AnimalFactory animalFactory = AnimalFactory.getAnimalFactoryInstance();
```

In this example, `AnimalFactory.getAnimalFactoryInstance()` would be implemented to return a static `AnimalFactory` object. This results in a single factory being instantiated and used rather than requiring a new factory to be instantiated each time the factory needs to be used.

Redo the `AnimalFactory` to make it a singleton, run the code and include you new modified code in your submission.

7.7.1.3 Abstract Factory Pattern



The abstract factory pattern is a creational design pattern. An abstract factory is a factory that returns factories. Why is this layer of abstraction useful? A normal factory can be used to create sets of related objects. An abstract factory returns factories. Thus, an abstract factory is used to return factories that can be used to create sets of related objects.

As an example, you could have a BMW factory that returns car part objects (seats, air filters, etc) associated with a BMW. You could also have a Toyota factory that returns car part objects associated with a Toyota. We could create an abstract factory that returns these different types of car factories depending on the car that we were interested in. We could then obtain car part objects from the car factory. Via polymorphism, we can use a common interface to get the different factories, and we could then use a common interface to get the different car parts.

Now, we'll look at the code for a simple example of the Abstract Factory Design Pattern.

Our AbstractFactory will return either a MammalFactory or a ReptileFactory via the SpeciesFactory return type. MammalFactory and ReptileFactory are subclasses of SpeciesFactory.

```
package za.ac.cput.kabasob.designpatterns.creational.abstractfactory;
/**
 *
 * @author boniface
 */
public class AbstractFactory {
    public SpeciesFactory getSpeciesFactory(String type) {
        if ("mammal".equals(type)) {
            return new MammalFactory();
        } else {
            return new ReptileFactory();
        }
    }
}
```

SpeciesFactory is an abstract class with the getAnimal() abstract method. This method returns an Animal object. The polymorphism of AbstractFactory is achieved because its getSpeciesFactory() method returns a SpeciesFactory, regardless of the actual underlying class. This polymorphism could also be achieved via an interface rather than an abstract class.

```
package za.ac.cput.kabasob.designpatterns.creational.abstractfactory;
/**
 *
 * @author boniface
 */
public abstract class SpeciesFactory {
    public abstract Animal getAnimal(String type);
}
```

MammalFactory implements getAnimal(). It returns an Animal, which is either a Dog or a Cat.

```
package za.ac.cput.kabasob.designpatterns.creational.abstractfactory;
/**
 *
 * @author boniface
 */
public class MammalFactory extends SpeciesFactory {
    @Override
    public Animal getAnimal(String type) {
        if ("dog".equals(type)) {
            return new Dog();
        }
    }
}
```

```

    } else {
        return new Cat();
    }
}

```

ReptileFactory implements getAnimal(). It returns an Animal, which is either a Snake or a Tyrannosaurus.

```

package za.ac.cput.kabasob.designpatterns.creational.abstractfactory;

/**
 *
 * @author boniface
 */
public class ReptileFactory extends SpeciesFactory {

    @Override
    public Animal getAnimal(String type) {
        if ("snake".equals(type)) {
            return new Snake();
        } else {
            return new Tyrannosaurus();
        }
    }
}

```

Animal is an abstract class with the makeSound() abstract method. Subclasses of Animal implement the makeSound() method.

```

package za.ac.cput.kabasob.designpatterns.creational.factorymethod;

/**
 *
 * @author boniface
 */
public abstract class Animal {
    public abstract String makeSound();
}

```

Cat Class

```

package za.ac.cput.kabasob.designpatterns.creational.factorymethod;

/**
 *
 * @author boniface
 */
public class Cat extends Animal {

    @Override
    public String makeSound() {
        return "Meow";
    }
}

```

The Dog class

```

package za.ac.cput.kabasob.designpatterns.creational.factorymethod;

/**
 *
 * @author boniface
 */
public class Dog extends Animal {

    @Override
    public String makeSound() {
        return "Woof";
    }
}

```

The Snake Class

```
package za.ac.cput.kabasob.designpatterns.creational.factorymethod;
/**
 *
 * @author boniface
 */
public class Cat extends Animal {
    @Override
    public String makeSound() {
        return "Hiss";
    }
}
```

The Tyrannosaurus class is shown here.

```
package za.ac.cput.kabasob.designpatterns.creational.factorymethod;
/**
 *
 * @author boniface
 */
public class Tyrannosaurus extends Animal {
    @Override
    public String makeSound() {
        return "Roar";
    }
}
```

The DriverMain class contains our main() method. It creates an AbstractFactory object. From the AbstractFactory, we obtain a SpeciesFactory (a ReptileFactory) and get two Animal objects (Tyrannosaurus and Snake) from the SpeciesFactory. After this, we obtain another SpeciesFactory (a MammalFactory) and then obtain two more Animal objects (Dog and Cat).

```
package za.ac.cput.kabasob.designpatterns.creational.abstractfactory;
/**
 *
 * @author boniface
 */
public abstract class MainDriver {
    public static void main(String[] args) {
        AbstractFactory abstractFactory = new AbstractFactory();

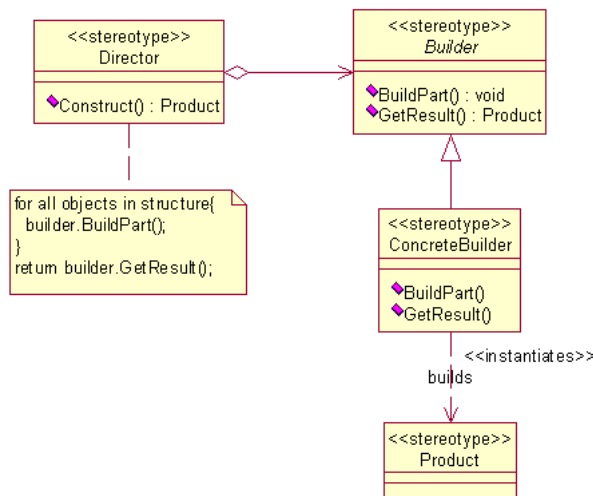
        SpeciesFactory speciesFactory1 = abstractFactory.getSpeciesFactory("reptile");
        Animal a1 = speciesFactory1.getAnimal("tyrannosaurus");
        System.out.println("a1 sound: " + a1.makeSound());
        Animal a2 = speciesFactory1.getAnimal("snake");
        System.out.println("a2 sound: " + a2.makeSound());

        SpeciesFactory speciesFactory2 = abstractFactory.getSpeciesFactory("mammal");
        Animal a3 = speciesFactory2.getAnimal("dog");
        System.out.println("a3 sound: " + a3.makeSound());
        Animal a4 = speciesFactory2.getAnimal("cat");
        System.out.println("a4 sound: " + a4.makeSound());
    }
}
```

Execute the code and record output in your log book.

Notice the use of polymorphism. We obtain different factories via the common SpeciesFactory superclass. We also obtain different animals via the common Animal superclass.

7.7.1.4 Builder Pattern



The builder pattern is a creational design pattern used to assemble complex objects. With the builder pattern, the same object construction process can be used to create different objects. The builder has 4 main parts: a Builder, Concrete Builders, a Director, and a Product.

A Builder is an interface (or abstract class) that is implemented (or extended) by Concrete Builders. The Builder interface sets forth the actions (methods) involved in assembling a Product object. It also has a method for retrieving the Product object (ie, `getProduct()`). The Product object is the object that gets assembled in the builder pattern.

Concrete Builders implement the Builder interface (or extend the Builder abstract class). A Concrete Builder is responsible for creating and assembling a Product object. Different Concrete Builders create and assemble Product objects differently.

A Director object is responsible for constructing a Product. It does this via the Builder interface to a Concrete Builder. It constructs a Product via the various Builder methods.

There are various uses of the builder pattern. For one, if we'd like the construction process to remain the same but we'd like to create a different type of Product, we can create a new Concrete Builder and pass this to the same Director. If we'd like to alter the construction process, we can modify the Director to use a different construction process.

Now, let's look at an example of the builder pattern. Our example will build different kinds of restaurant meals.

First off, our Product will be a Meal class, which represents food items in a meal. It represents a drink, main course, and side item.

```

package za.ac.cput.kabasob.designpatterns.creational.builder;

/**
 *
 * @author boniface
 */
public class Meal {

```

```

private String drink;
private String mainCourse;
private String side;

public String getDrink() {
    return drink;
}

public void setDrink(String drink) {
    this.drink = drink;
}

public String getMainCourse() {
    return mainCourse;
}

public void setMainCourse(String mainCourse) {
    this.mainCourse = mainCourse;
}

public String getSide() {
    return side;
}

public void setSide(String side) {
    this.side = side;
}

public String toString() {
    return "drink:" + drink + ", main course:" + mainCourse + ", side:" + side;
}
}

```

Our Builder interface is MealBuilder. It features methods used to build a meal and a method to retrieve the meal.

```

package za.ac.cput.kabasob.designpatterns.creational.builder;

/**
 *
 * @author boniface
 */
public interface MealBuilder {
    public void buildDrink();

    public void buildMainCourse();

    public void buildSide();

    public Meal getMeal();
}

```

Our first Concrete Builder is ItalianMealBuilder. Its constructor creates a meal. Its methods are implemented to build the various parts of the meal. It returns the meal via getMeal().

```

package za.ac.cput.kabasob.designpatterns.creational.builder;

/**
 *
 * @author boniface
 */
public class ItalianMealBuilder implements MealBuilder {

    private Meal meal;

    public ItalianMealBuilder() {
        meal = new Meal();
    }

    @Override
    public void buildDrink() {
        meal.setDrink("red wine");
    }

    @Override
    public void buildMainCourse() {
        meal.setMainCourse("pizza");
    }

    @Override
    public void buildSide() {
        meal.setSide("bread");
    }
}

```

```
@Override
public Meal getMeal() {
    return meal;
}
```

Our second Concrete Builder is JapaneseMealBuilder. Its constructor creates a meal. Its methods are implemented to build the various parts of a Japanese meal. It returns the meal via getMeal().

```
package za.ac.cput.kabasob.designpatterns.creational.builder;

/**
 * @author boniface
 */
public class JapaneseMealBuilder implements MealBuilder {

    private Meal meal;

    public JapaneseMealBuilder() {
        meal = new Meal();
    }

    @Override
    public void buildDrink() {
        meal.setDrink("sake");
    }

    @Override
    public void buildMainCourse() {
        meal.setMainCourse("chicken teriyaki");
    }

    @Override
    public void buildSide() {
        meal.setSide("miso soup");
    }

    @Override
    public Meal getMeal() {
        return meal;
    }
}
```

Our Director class is MealDirector. It takes a MealBuilder as a parameter in its constructor. Thus, a different type of meal will be assembled by the MealDirector depending on the Concrete Builder passed in to the constructor. The assembly of the meal (Product) takes place in the constructMeal() method of the Director. This method spells out the parts of the meal that will be assembled.

```
package za.ac.cput.kabasob.designpatterns.creational.builder;

/**
 * @author boniface
 */
public class MealDirector {

    private MealBuilder mealBuilder = null;

    public MealDirector(MealBuilder mealBuilder) {
        this.mealBuilder = mealBuilder;
    }

    public void constructMeal() {
        mealBuilder.buildDrink();
        mealBuilder.buildMainCourse();
        mealBuilder.buildSide();
    }

    public Meal getMeal() {
        return mealBuilder.getMeal();
    }
}
```

The MainDriver class lets us demonstrate our builder pattern. First, our director builds an Italian meal. An ItalianMealBuilder is passed to the MealDirector's constructor. The meal is constructed via mealDirector.constructMeal(). The meal is obtained from

mealDirector via mealDirector.getMeal(). The Italian meal is displayed. After this, we perform the same process to build and display a Japanese meal.

```
package za.ac.cput.kabasob.designpatterns.creational.builder;

/**
 *
 * @author boniface
 */
public class MainDriver {

    public static void main(String[] args) {

        MealBuilder mealBuilder = new ItalianMealBuilder();
        MealDirector mealDirector = new MealDirector(mealBuilder);
        mealDirector.constructMeal();
        Meal meal = mealDirector.getMeal();
        System.out.println("meal is: " + meal);

        mealBuilder = new JapaneseMealBuilder();
        mealDirector = new MealDirector(mealBuilder);
        mealDirector.constructMeal();
        meal = mealDirector.getMeal();
        System.out.println("meal is: " + meal);

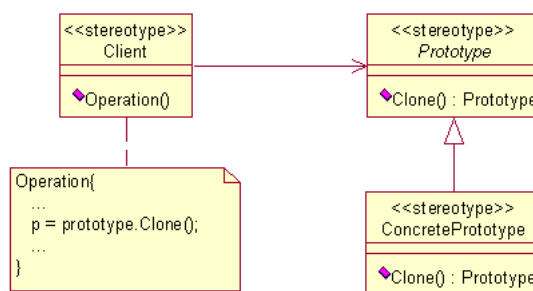
    }
}
```

Record Output in your log book

Notice that if we'd like to create a new type of meal, we can do so by implementing a new Concrete Builder (ie, SwedishMealBuilder, FrenchMealBuilder, etc), which we'd pass to the MealDirector. If we'd like the meal to be constructed of different parts (ie, no drink), we can alter the construction process in MealDirector. Thus, construction process has been separated to the Director, and the data representation is controlled by the Concrete Builders that implement the Builder interface.

Create a two extra meals for your Example and include the code in your submission

7.7.1.5 Prototype Pattern



The prototype pattern is a creational design pattern. In the prototype pattern, a new object is created by cloning an existing object. In Java, the clone() method is an implementation of this design pattern. The prototype pattern can be a useful way of creating copies of objects.

One example of how this can be useful is if an original object is created with a resource such as a data stream that may not be available at the time that a clone of the object is needed.

Another example is if the original object creation involves a significant time commitment, such as reading data from a database or over a network. An added benefit of the

prototype pattern is that it can reduce class proliferation in a project by avoiding factory proliferation.

Normally in Java, if you'd like to use cloning (ie, the prototype pattern), you can utilize the clone() method and the Cloneable interface. By default, clone() performs a shallow copy

However, we can implement our own prototype pattern. To do so, we'll create a Prototype interface that features a doClone() method.

```
public interface Prototype {  
    public Prototype doClone();  
}
```

The Person class implements the doClone() method. This method creates a new Person object and clones the name field. It returns the newly cloned Person object.

```
package za.ac.cput.kabasob.designpatterns.creational.prototype;  
/**  
 *  
 * @author boniface  
 */  
public class Person implements Prototype {  
    String name;  
  
    public Person(String name) {  
        this.name = name;  
    }  
  
    @Override  
    public Prototype doClone() {  
        return new Person(name);  
    }  
  
    public String toString() {  
        return "This person is named " + name;  
    }  
}
```

The Dog class also implements the doClone() method. This method creates a new Dog object and clones the sound field. The cloned Dog object is returned.

```
package za.ac.cput.kabasob.designpatterns.creational.prototype;  
/**  
 *  
 * @author boniface  
 */  
public class Dog implements Prototype {  
    String sound;  
  
    public Dog(String sound) {  
        this.sound = sound;  
    }  
  
    @Override  
    public Prototype doClone() {  
        return new Dog(sound);  
    }  
  
    public String toString() {  
        return "This dog says " + sound;  
    }  
}
```

The MainDriver creates a Person object and then clones it to a second Person object. It then creates a Dog object and clones it to a second Dog object.

```
package za.ac.cput.kabasob.designpatterns.creational.prototype;

/**
 *
 * @author boniface
 */
public class MainDriver {

    public static void main(String[] args) {

        Person person1 = new Person("Fred");
        System.out.println("person 1:" + person1);
        Person person2 = (Person) person1.doClone();
        System.out.println("person 2:" + person2);

        Dog dog1 = new Dog("Woof!");
        System.out.println("dog 1:" + dog1);
        Dog dog2 = (Dog) dog1.doClone();
        System.out.println("dog 2:" + dog2);

    }
}
```

Record output in your log book

7.7.2 Structural Design Patterns

In software engineering, structural design patterns are design patterns that ease the design by identifying a simple way to realize relationships between entities.

A structural design pattern serves as a blueprint for how different classes and objects are combined to form larger structures. Unlike creational patterns, which are mostly different ways to fulfill the same fundamental purpose, each structural pattern has a different purpose.

Structural Patterns describe how objects and classes can be combined to form structures. We distinguish between object patterns and class patterns.

The difference is that class patterns describe relationships and structures with the help of inheritance. Object patterns, on other hand, describe how objects can be associated and aggregated to form larger, more complex structures.

In some sense, structural patterns are similar to the simpler concept of data structures.

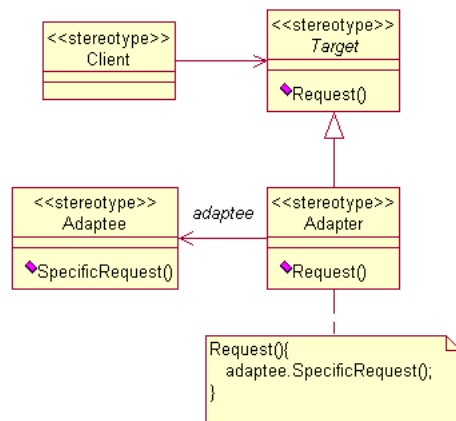
However, structural design patterns also specify the methods that connect objects, not merely the references between them. Furthermore, data structures are fairly static entities. They only describe how data is arranged in the structure.

A structural design pattern also describes how data moves through the pattern. Structural class patterns use inheritance to combine the interfaces or implementations of multiple classes.

Structural object patterns use object composition to combine the implementations of multiple objects. They can combine the interfaces of all the composed objects into one unified interface or they can provide a completely new interface.

1. Adapter pattern
2. Composite pattern
3. Proxy pattern
4. Flyweight Pattern
5. Facade Pattern
6. Bridge Pattern
7. Decorator Pattern

7.7.2.1 Adapter Pattern



The adapter pattern is a structural design pattern. In the Adapter pattern, a wrapper class (ie, the adapter) is used to translate requests from it to another class (ie, the adaptee). In effect, an adapter provides particular interactions with an adaptee that are not offered directly by the adaptee.

The adapter pattern takes two forms. In the first form, a "class adapter" utilizes inheritance. The class adapter extends the adaptee class and adds the desired methods to the adapter. These methods can be declared in an interface (ie, the "target" interface).

In the second form, an "object adapter" utilizes composition. The object adapter contains an adaptee and implements the target interface to interact with the adaptee.

Now let's look at simple examples of a class adapter and an object adapter. First, we have an adaptee class named CelciusReporter. It stores a temperature value in Celcius.

```

package za.ac.cput.kabasob.designpatterns.structural.adapter;

/**
 *
 * @author boniface
 */
public class CelciusReporter {
    double temperatureInC;

    public CelciusReporter() {
    }

    public double getTemperature() {
        return temperatureInC;
    }

    public void setTemperature(double temperatureInC) {
        this.temperatureInC = temperatureInC;
    }
}
  
```

Here is our target interface that will be implemented by our adapter. It defines actions that our adapter will perform.

```

package za.ac.cput.kabasob.designpatterns.structural.adapter;

/**
 *
 * @author boniface
 */
public interface TemperatureInfo {
  }
  
```

```

    public double getTemperatureInF();

    public void setTemperatureInF(double temperatureInF);

    public double getTemperatureInC();

    public void setTemperatureInC(double temperatureInC);
}

```

TemperatureClassReporter is a class adapter. It extends CelciusReporter (the adaptee) and implements TemperatureInfo (the target interface). If a temperature is in Celcius, TemperatureClassReporter utilizes the temperatureInC value from CelciusReporter. Fahrenheit requests are internally handled in Celcius.

```

package za.ac.cput.kabasob.designpatterns.structural.adapter;

/**
 *
 * @author boniface
 */
// example of a class adapter
public class TemperatureClassReporter extends CelciusReporter implements TemperatureInfo {

    @Override
    public double getTemperatureInC() {
        return temperatureInC;
    }

    @Override
    public double getTemperatureInF() {
        return cToF(temperatureInC);
    }

    @Override
    public void setTemperatureInC(double temperatureInC) {
        this.temperatureInC = temperatureInC;
    }

    @Override
    public void setTemperatureInF(double temperatureInF) {
        this.temperatureInC = fToC(temperatureInF);
    }

    private double fToC(double f) {
        return ((f - 32) * 5 / 9);
    }

    private double cToF(double c) {
        return ((c * 9 / 5) + 32);
    }
}

```

TemperatureObjectReporter is an object adapter. It is similar in functionality to TemperatureClassReporter, except that it utilizes composition for the CelciusReporter rather than inheritance.

```

package za.ac.cput.kabasob.designpatterns.structural.adapter;

/**
 *
 * @author boniface
 */
// example of an object adapter
public class TemperatureObjectReporter implements TemperatureInfo {

    CelciusReporter celciusReporter;

    public TemperatureObjectReporter() {
        celciusReporter = new CelciusReporter();
    }

    @Override
    public double getTemperatureInC() {
        return celciusReporter.getTemperature();
    }

    @Override
    public double getTemperatureInF() {
        return cToF(celciusReporter.getTemperature());
    }
}

```

```

@Override
public void setTemperatureInC(double temperatureInC) {
    celciusReporter.setTemperature(temperatureInC);
}

@Override
public void setTemperatureInF(double temperatureInF) {
    celciusReporter.setTemperature(fToC(temperatureInF));
}

private double fToC(double f) {
    return ((f - 32) * 5 / 9);
}

private double cToF(double c) {
    return ((c * 9 / 5) + 32);
}
}

```

The MainDriver class is a client class that demonstrates the adapter pattern. First, it creates a TemperatureClassReporter object and references it via a TemperatureInfo reference. It demonstrates calls to the class adapter via the TemperatureInfo interface. After this, it creates a TemperatureObjectReporter object and references it via the same TemperatureInfo reference. It then demonstrates calls to the object adapter.

```

package za.ac.cput.kabasob.designpatterns.structural.adapter;

/**
 * @author boniface
 */
public class DriverMain {

    public static void main(String[] args) {

        // class adapter
        System.out.println("class adapter test");
        TemperatureInfo tempInfo = new TemperatureClassReporter();
        testTempInfo(tempInfo);

        // object adapter
        System.out.println("\nobject adapter test");
        tempInfo = new TemperatureObjectReporter();
        testTempInfo(tempInfo);

    }

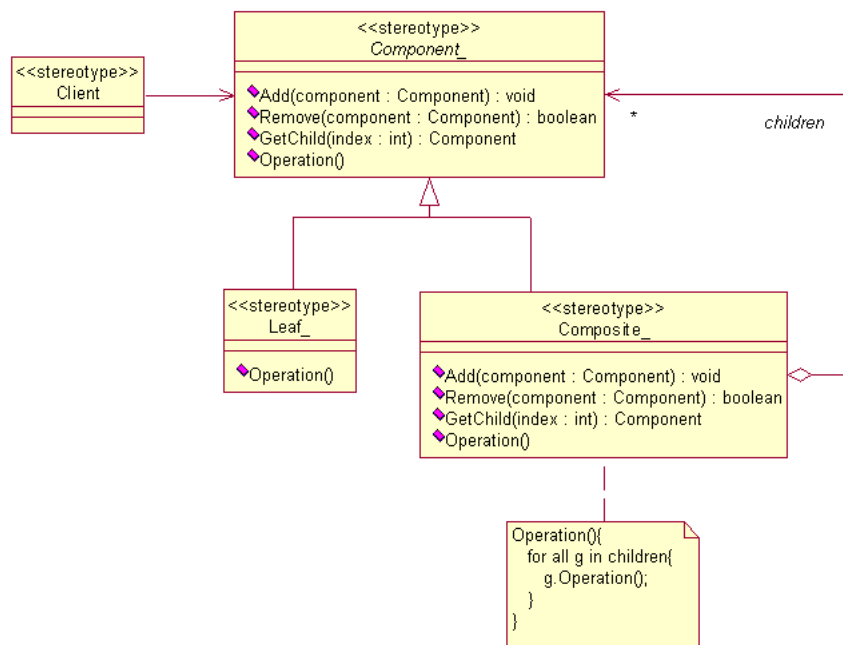
    public static void testTempInfo(TemperatureInfo tempInfo) {
        tempInfo.setTemperatureInC(0);
        System.out.println("temp in C:" + tempInfo.getTemperatureInC());
        System.out.println("temp in F:" + tempInfo.getTemperatureInF());

        tempInfo.setTemperatureInF(85);
        System.out.println("temp in C:" + tempInfo.getTemperatureInC());
        System.out.println("temp in F:" + tempInfo.getTemperatureInF());
    }
}

```

Write output in your log book

7.7.2.2 Composite Pattern



The composite pattern is a structural design pattern. In the composite pattern, a tree structure exists where identical operations can be performed on leaves and nodes. A node in a tree is a class that can have children. A node class is a 'composite' class. A leaf in a tree is a 'primitive' class that does not have children. The children of a composite can be leaves or other composites.

The leaf class and the composite class share a common 'component' interface that defines the common operations that can be performed on leaves and composites. When an operation on a composite is performed, this operation is performed on all children of the composite, whether they are leaves or composites. Thus, the composite pattern can be used to perform common operations on the objects that compose a tree.

The Gang of Four description of the composite pattern defines a client's interaction with the tree structure via a Component interface, where this interface includes the common operations on the composite and leaf classes, and the add/remove/get operations on the composites of the tree. This seems slightly awkward since a leaf does not implement the add/remove/get operations. In Java it makes more sense to define the common leaf/composite operations on a Component interface, but to put the add/remove/get composite operations in a separate interface or to simply implement them in the composite class.

Now we'll look at an example of the composite pattern. First, we'll declare a Component interface that declares the operations that are common for the composite class and the leaf class. This allows us to perform operations on composites and leaves using one standard interface.

```

package za.ac.cput.kabasob.designpatterns.structural.composite;

/**
 *
 * @author boniface
 */

```

```
*/  
public interface Component {  
    public void sayHello();  
    public void sayGoodbye();  
}
```

The Leaf class has a name field and implements the sayHello() and sayGoodbye() methods of the Component interface by outputting messages to standard output.

```
package za.ac.cput.kabasob.designpatterns.structural.composite;  
  
/**  
 *  
 * @author boniface  
 */  
public class Leaf implements Component {  
    String name;  
  
    public Leaf(String name) {  
        this.name = name;  
    }  
  
    @Override  
    public void sayHello() {  
        System.out.println(name + " leaf says hello");  
    }  
  
    @Override  
    public void sayGoodbye() {  
        System.out.println(name + " leaf says goodbye");  
    }  
}
```

The Composite class implements the Component interface. It implements the sayHello() and sayGoodbye() methods by calling these same methods on all of its children, which are Components (since they can be both Leaf objects and Composite objects, which both implement the Component interface).

```
package za.ac.cput.kabasob.designpatterns.structural.composite;  
  
import java.util.ArrayList;  
import java.util.List;  
  
/**  
 *  
 * @author boniface  
 */  
public class Composite implements Component {  
    List<Component> components = new ArrayList<Component>();  
  
    @Override  
    public void sayHello() {  
        for (Component component : components) {  
            component.sayHello();  
        }  
    }  
  
    @Override  
    public void sayGoodbye() {  
        for (Component component : components) {  
            component.sayGoodbye();  
        }  
    }  
  
    public void add(Component component) {  
        components.add(component);  
    }  
  
    public void remove(Component component) {  
        components.remove(component);  
    }  
  
    public List<Component> getComponents() {  
        return components;  
    }  
  
    public Component getComponent(int index) {  
        return components.get(index);  
    }  
}
```



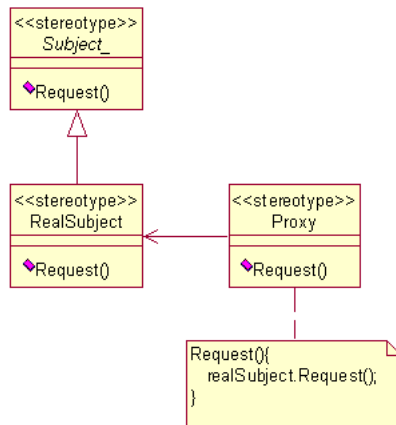
```
}  
}
```

The MainDriver class demonstrates the composite pattern. It creates 5 Leaf objects. It adds two of these to a Composite object and two of these to another Composite object. It adds these two Composite objects and the last Leaf object to another Composite object. It calls sayHello() on leaf1, then sayHello() on composite1, then sayHello() on composite2, and then sayGoodbye() on composite3.

```
package za.ac.cput.kabasob.designpatterns.structural.composite;  
  
/**  
 *  
 * @author boniface  
 */  
public class MainDriver {  
  
    public static void main(String[] args) {  
  
        Leaf leaf1 = new Leaf("Bob");  
        Leaf leaf2 = new Leaf("Fred");  
        Leaf leaf3 = new Leaf("Sue");  
        Leaf leaf4 = new Leaf("Ellen");  
        Leaf leaf5 = new Leaf("Joe");  
  
        Composite composite1 = new Composite();  
        composite1.add(leaf1);  
        composite1.add(leaf2);  
  
        Composite composite2 = new Composite();  
        composite2.add(leaf3);  
        composite2.add(leaf4);  
  
        Composite composite3 = new Composite();  
        composite3.add(composite1);  
        composite3.add(composite2);  
        composite3.add(leaf5);  
  
        System.out.println("Calling sayHello on leaf1");  
        leaf1.sayHello();  
  
        System.out.println("\nCalling sayHello on composite1");  
        composite1.sayHello();  
  
        System.out.println("\nCalling sayHello on composite2");  
        composite2.sayHello();  
  
        System.out.println("\nCalling sayGoodbye on composite3");  
        composite3.sayGoodbye();  
  
    }  
}
```

Write your output in the log book

7.7.2.3 Proxy Pattern



The proxy pattern is a structural design pattern. In the proxy pattern, a proxy class is used to control access to another class. The reasons for this control can vary. As one example, a proxy may avoid instantiation of an object until the object is needed. This can be useful if the object requires a lot of time or resources to create. Another reason to use a proxy is to control access rights to an object. A client request may require certain credentials in order to access the object.

Now, we'll look at an example of the proxy pattern. First, we'll create an abstract class called Thing with a basic sayHello() message that includes the date/time that the message is displayed.

```

package za.ac.cput.kabasob.designpatterns.structural.proxy;

import java.util.Date;

/**
 *
 * @author boniface
 */
public abstract class Thing {

    public void sayHello() {
        System.out.println(this.getClass().getSimpleName() + " says howdy at " + new Date());
    }
}
  
```

FastThing subclasses Thing.

```

package za.ac.cput.kabasob.designpatterns.structural.proxy;

/**
 *
 * @author boniface
 */
public class FastThing extends Thing {

    public FastThing() {
    }
}
  
```

SlowThing also subclasses Thing. However, its constructor takes 5 seconds to execute.

```

package za.ac.cput.kabasob.designpatterns.structural.proxy;
  
```

```
/**
 *
 * @author boniface
 */
public class SlowThing extends Thing {

    public SlowThing() {
        try {
            Thread.sleep(5000);
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    }
}
```

The Proxy class is a proxy to a SlowThing object. Since a SlowThing object takes 5 seconds to create, we'll use a proxy to a SlowThing so that a SlowThing object is only created on demand. This occurs when the proxy's sayHello() method is executed. It instantiates a SlowThing object if it doesn't already exist and then calls sayHello() on the SlowThing object.

```
package za.ac.cput.kabasob.designpatterns.structural.proxy;

import java.util.Date;

/**
 *
 * @author boniface
 */
public class Proxy {

    SlowThing slowThing;

    public Proxy() {
        System.out.println("Creating proxy at " + new Date());
    }

    public void sayHello() {
        if (slowThing == null) {
            slowThing = new SlowThing();
        }
        slowThing.sayHello();
    }
}
```

The MainDriver class demonstrates the use of our proxy. It creates a Proxy object and then creates a FastThing object and calls sayHello() on the FastThing object. Next, it calls sayHello() on the Proxy object.

```
package za.ac.cput.kabasob.designpatterns.structural.proxy;

/**
 *
 * @author boniface
 */
public class MainDriver {

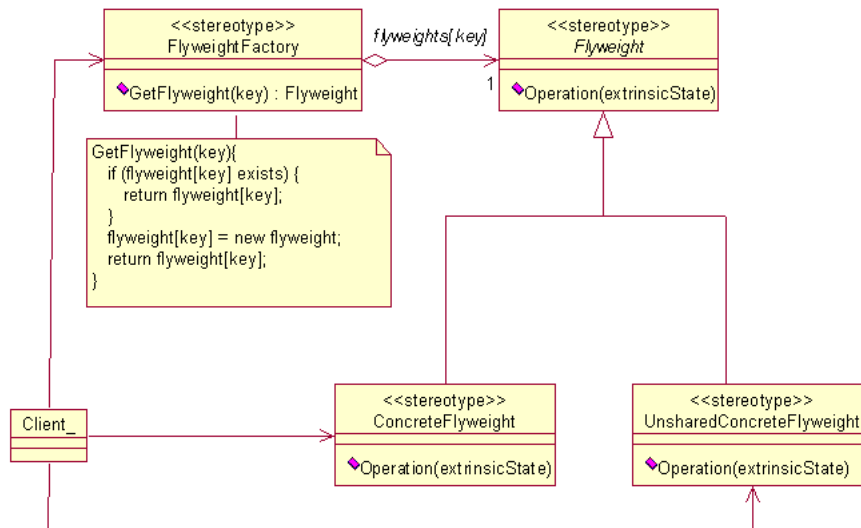
    public static void main(String[] args) {
        Proxy proxy = new Proxy();
        FastThing fastThing = new FastThing();
        fastThing.sayHello();
        proxy.sayHello();
    }
}
```

Record your output in the log book.

From the output, notice that creating the Proxy object and calling sayHello() on the FastThing object occurred at a specific time, and calling sayHello() on the Proxy object did not call sayHello() on the SlowThing object until another time. We can see that the SlowThing creation was a time-consuming process. However, this did not slow down the execution of our application until the SlowThing object was actually required. We can

see here that the proxy pattern avoids the creation of time-consuming objects until they are actually needed.

7.7.2.4 Flyweight Pattern



The flyweight pattern is a structural design pattern. In the flyweight pattern, instead of creating large numbers of similar objects, objects are reused. This can be used to reduce memory requirements and instantiation time and related costs. Similarities between objects are stored inside of the objects, and differences are moved outside of the objects and placed in client code. These differences are passed in to the objects when needed via method calls on the objects. A Flyweight interface declares these methods. A Concrete Flyweight class implements the Flyweight interface which is used to perform operations based on external state and it also stores common state. A Flyweight Factory is used to create and return Flyweight objects.

Now, let's look at an example of the flyweight design pattern. We'll create a Flyweight interface with a doMath() method that will be used to perform a mathematical operation on two integers passed in as parameters.

```

package za.ac.cput.kabasob.designpatterns.structural.flyweight;

/**
 * @author boniface
 */
public interface Flyweight {

    public void doMath(int a, int b);

}
  
```

The FlyweightAdder class is a concrete flyweight class. It contains an "operation" field that is used to store the name of an operation that is common to adder flyweights. Notice the call to Thread.sleep(3000). This simulates a construction process that is expensive in terms of time. Each FlyweightAdder object that is created takes 3 seconds to create, so we definitely want to minimize the number of flyweight objects that are created. The doMath() method is implemented. It displays the common "operation"

field and displays the addition of a and b, which are external state values that are passed in and used by the FlyweightAdder when doMath() is executed.

```
package za.ac.cput.kabasob.designpatterns.structural.flyweight;

/**
 *
 * @author boniface
 */
public class FlyweightAdder implements Flyweight {

    String operation;

    public FlyweightAdder() {
        operation = "adding";
        try {
            Thread.sleep(3000);
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    }

    @Override
    public void doMath(int a, int b) {
        System.out.println(operation + " " + a + " and " + b + ": " + (a + b));
    }
}
```

The FlyweightMultiplier class is similar to the FlyweightAdder class, except that it performs multiplication rather than addition.

```
package za.ac.cput.kabasob.designpatterns.structural.flyweight;

/**
 *
 * @author boniface
 */
public class FlyweightMultiplier implements Flyweight {

    String operation;

    public FlyweightMultiplier() {
        operation = "multiplying";
        try {
            Thread.sleep(3000);
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    }

    @Override
    public void doMath(int a, int b) {
        System.out.println(operation + " " + a + " and " + b + ": " + (a * b));
    }
}
```

The FlyweightFactory class is our flyweight factory. It utilizes the singleton pattern so that we only have once instance of the factory, which we obtain via its static getInstance() method. The FlyweightFactory creates a hashmap pool of flyweights. If a request is made for a flyweight object and that object doesn't exist, it is created and placed in the flyweight pool. The flyweight pool of the FlyweightFactory stores all the instances of the different types of flyweights (ie, FlyweightAdder object, FlyweightMultiplier object, etc). Thus, only one instance of each type is created, and this occurs on-demand.

```
package za.ac.cput.kabasob.designpatterns.structural.flyweight;

import java.util.HashMap;
import java.util.Map;

/**
 *
 * @author boniface
 */
public class FlyweightFactory {
```

```

private static FlyweightFactory flyweightFactory;
private Map<String, Flyweight> flyweightPool;

private FlyweightFactory() {
    flyweightPool = new HashMap<String, Flyweight>();
}
public static FlyweightFactory getInstance() {
    if (flyweightFactory == null) {
        flyweightFactory = new FlyweightFactory();
    }
    return flyweightFactory;
}

public Flyweight getFlyweight(String key) {
    if (flyweightPool.containsKey(key)) {
        return flyweightPool.get(key);
    } else {
        Flyweight flyweight;
        if ("add".equals(key)) {
            flyweight = new FlyweightAdder();
        } else {
            flyweight = new FlyweightMultiplier();
        }
        flyweightPool.put(key, flyweight);
        return flyweight;
    }
}
}

```

The MainDriver class demonstrates our flyweight pattern. It obtains a FlyweightFactory object via FlyweightFactory.getInstance(). After this, in a loop, it obtains a FlyweightAdder from the FlyweightFactory and calls its doMath() operation with the current loop index as the two parameter values. Next, it does the same thing with a FlyweightMultiplier.

```

package za.ac.cput.kabasob.designpatterns.structural.flyweight;

/**
 *
 * @author boniface
 */
public class MainDriver {

    public static void main(String[] args) {

        FlyweightFactory flyweightFactory = FlyweightFactory.getInstance();

        for (int i = 0; i < 5; i++) {
            Flyweight flyweightAdder = flyweightFactory.getFlyweight("add");
            flyweightAdder.doMath(i, i);

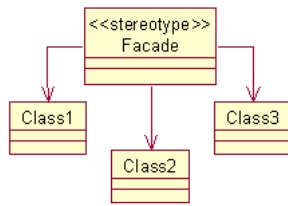
            Flyweight flyweightMultiplier = flyweightFactory.getFlyweight("multiply");
            flyweightMultiplier.doMath(i, i);
        }
    }
}

```

Write output in your log book

When executing MainDriver, we see that it takes some seconds to create the FlyweightAdder object and the same thing happens for the FlyweightMultiplier object. After this, no more delays occur since we keep reusing the flyweight objects rather than instantiating new objects. This highlights how the flyweight pattern can be used to minimize resource requirements by avoiding unnecessary object instantiations for similar objects.

7.7.2.5 Facade Pattern



The facade pattern is a structural design pattern. In the facade pattern, a facade classes is used to provide a single interface to set of classes. The Facade simplifies a clients interaction with a complex system by localizing the interactions into a single interface. As a result, the client can interact with a single object rather than being required to interact directly in complicated ways with the objects that make up the subsystem.

As an example, supposed we have three horribly written classes. For based on the class and method names (and the lack of documentation), it would be very difficult for a client to interact with these classes.

Class1's `doSomethingComplicated()` method takes an integer and returns its cube.

```
package za.ac.cput.kabasob.designpatterns.structural.facade;

/**
 *
 * @author boniface
 */
public class Class1 {

    public int doSomethingComplicated(int x) {
        return x * x * x;
    }
}
```

Class2's `doAnotherThing()` method doubles the cube of an integer and returns it.

```
package za.ac.cput.kabasob.designpatterns.structural.facade;

/**
 *
 * @author boniface
 */
public class Class2 {

    public int doAnotherThing(Class1 class1, int x) {
        return 2 * class1.doSomethingComplicated(x);
    }
}
```

Class3's `doMoreStuff()` takes a Class1 object, a Class2 object, and an integer and returns twice the sixth power of the integer.

```
package za.ac.cput.kabasob.designpatterns.structural.facade;

/**
 *
 * @author boniface
 */
public class Class3 {

    public int doMoreStuff(Class1 class1, Class2 class2, int x) {
        return class1.doSomethingComplicated(x) * class2.doAnotherThing(class1, x);
    }
}
```

For a client unfamiliar with Class1, Class2, and Class3, it would be very difficult to figure out how to interact with these classes. The classes interact and perform tasks in unclear ways. As a result, we need to be able to simplify interaction with this system of classes so that clients can interact with these classes in a simple, standardized manner.

We do this with the Facade class. The Facade class has three methods: cubeX(), cubeXTimes2(), and xToSixthPowerTimes2(). The names of these methods clearly indicate what they do, and these methods hide the interactions of Class1, Class2, and Class3 from client code.

```
package za.ac.cput.kabasob.designpatterns.structural.facade;

/**
 * @author boniface
 */
public class Facade {

    public int cubeX(int x) {
        Class1 class1 = new Class1();
        return class1.doSomethingComplicated(x);
    }

    public int cubeXTimes2(int x) {
        Class1 class1 = new Class1();
        Class2 class2 = new Class2();
        return class2.doAnotherThing(class1, x);
    }

    public int xToSixthPowerTimes2(int x) {
        Class1 class1 = new Class1();
        Class2 class2 = new Class2();
        Class3 class3 = new Class3();
        return class3.doMoreStuff(class1, class2, x);
    }
}
```

The MainDriver class contains our client code. It creates a Facade object and then calls its three methods with a parameter value of 3. It displays the returned results.

```
package za.ac.cput.kabasob.designpatterns.structural.facade;

/**
 * @author boniface
 */
public class MainDriver {

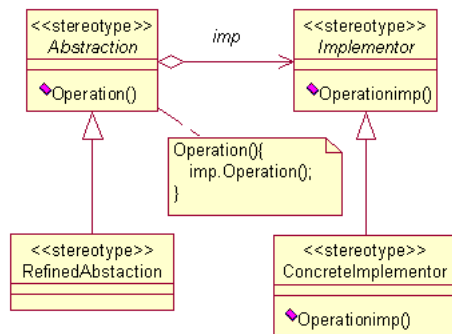
    public static void main(String[] args) {

        Facade facade = new Facade();
        int x = 3;
        System.out.println("Cube of " + x + ":" + facade.cubeX(3));
        System.out.println("Cube of " + x + " times 2:" + facade.cubeXTimes2(3));
        System.out.println(x + " to sixth power times 2:" + facade.xToSixthPowerTimes2(3));
    }
}
```

Record your output in your log book

This example demonstrates how the facade pattern can be used to simplify interactions with a system of classes by providing a single point of interaction with the subsystem and hiding the complex details of subsystem interactions from client code. This is accomplished with a Facade class.

7.7.2.6 Bridge Pattern



The bridge pattern is a structural design pattern. In the bridge pattern, we separate an abstraction and its implementation and develop separate inheritance structures for both the abstraction and the implementor. The abstraction is an interface or abstract class, and the implementor is likewise an interface or abstract class. The abstraction contains a reference to the implementor. Children of the abstraction are referred to as refined abstractions, and children of the implementor are concrete implementors. Since we can change the reference to the implementor in the abstraction, we are able to change the abstraction's implementor at run-time. Changes to the implementor do not affect client code.

The bridge pattern can be demonstrated with an example. Suppose we have a Vehicle class. We can extract out the implementation of the engine into an Engine class. We can reference this Engine implementor in our Vehicle via an Engine field. We'll declare Vehicle to be an abstract class. Subclasses of Vehicle need to implement the drive() method. Notice that the Engine reference can be changed via the setEngine() method.

```

package za.ac.cput.kabasob.designpatterns.structural.bridge;

/**
 *
 * @author boniface
 */
public abstract class Vehicle {

    Engine engine;
    int weightInKilos;

    public abstract void drive();

    public void setEngine(Engine engine) {
        this.engine = engine;
    }

    public void reportOnSpeed(int horsepower) {
        int ratio = weightInKilos / horsepower;
        if (ratio < 3) {
            System.out.println("The vehicle is going at a fast speed.");
        } else if ((ratio >= 3) && (ratio < 8)) {
            System.out.println("The vehicle is going an average speed.");
        } else {
            System.out.println("The vehicle is going at a slow speed.");
        }
    }
}
  
```

BigBus is a subclass of Vehicle. It has a weight of 3000 kg. Its drive() method displays a message, calls the engine's go() method, and then calls reportOnSpeed() with the horsepower of the engine to report on how fast the vehicle is moving.

```
package za.ac.cput.kabasob.designpatterns.structural.bridge;

/**
 *
 * @author boniface
 */
public class BigBus extends Vehicle {

    public BigBus(Engine engine) {
        this.weightInKilos = 3000;
        this.engine = engine;
    }

    @Override
    public void drive() {
        System.out.println("\nThe big bus is driving");
        int horsepower = engine.go();
        reportOnSpeed(horsepower);
    }

}
```

SmallCar is similar to BigBus but is much lighter.

```
package za.ac.cput.kabasob.designpatterns.structural.bridge;

/**
 *
 * @author boniface
 */
public class SmallCar extends Vehicle {

    public SmallCar(Engine engine) {
        this.weightInKilos = 600;
        this.engine = engine;
    }

    @Override
    public void drive() {
        System.out.println("\nThe small car is driving");
        int horsepower = engine.go();
        reportOnSpeed(horsepower);
    }

}
```

Our implementor interface is the Engine interface, which declares the go() method.

```
package za.ac.cput.kabasob.designpatterns.structural.bridge;

/**
 *
 * @author boniface
 */
public interface Engine {
    public int go();
}
```

```
package za.ac.cput.kabasob.designpatterns.structural.bridge;

/**
 *
 * @author boniface
 */
public class BigEngine implements Engine {

    int horsepower;

    public BigEngine() {
        horsepower = 350;
    }

    @Override
    public int go() {
        System.out.println("The big engine is running");
        return horsepower;
    }

}
```

```
package za.ac.cput.kabasob.designpatterns.structural.bridge;

/**
 *
 * @author boniface
 */
public class SmallEngine implements Engine {

    int horsepower;

    public SmallEngine() {
        horsepower = 100;
    }

    @Override
    public int go() {
        System.out.println("The small engine is running");
        return horsepower;
    }
}
```

The MainDriver class demonstrates our bridge pattern. We create a BigBus vehicle with a SmallEngine implementor. We call the vehicle's drive() method. Next, we change the implementor to a BigEngine and once again call drive(). After this, we create a SmallCar vehicle with a SmallEngine implementor. We call drive(). Next, we change the engine to a BigEngine and once again call drive().

```
package za.ac.cput.kabasob.designpatterns.structural.bridge;

/**
 *
 * @author boniface
 */
public class MainDriver {

    public static void main(String[] args) {

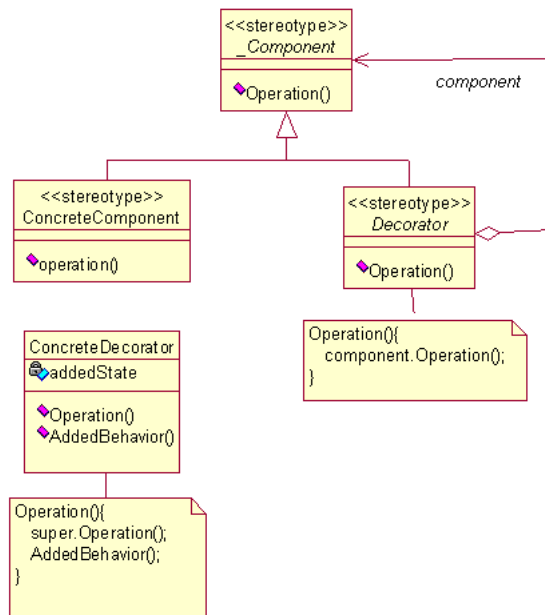
        Vehicle vehicle = new BigBus(new SmallEngine());
        vehicle.drive();
        vehicle.setEngine(new BigEngine());
        vehicle.drive();

        vehicle = new SmallCar(new SmallEngine());
        vehicle.drive();
        vehicle.setEngine(new BigEngine());
        vehicle.drive();
    }
}
```

Record your output in the lab book

Notice that we were able to change the implementor (engine) dynamically for each vehicle. These changes did not affect the client code in MainDriver. In addition, since BigBus and SmallCar were both subclasses of the Vehicle abstraction, we were even able to point the vehicle reference to a BigBus object and a SmallCar object and call the same drive() method for both types of vehicles.

7.7.2.7 Decorator Pattern



The decorator pattern is a structural design pattern. Whereas inheritance adds functionality to classes, the decorator pattern adds functionality to objects by wrapping objects in other objects. Each time additional functionality is required, the object is wrapped in another object. Java I/O streams are a well-known example of the decorator pattern.

For the decorator pattern, we require a class that serves as the base object that we add functionality to. This is a Concrete Component, and it implements a Component interface. The Component interface declares the common operations that are to be performed by the concrete component and all the decorators that wrap the concrete component object. A Decorator is an abstract class that implements the Component interface and contains a reference to a Component. Concrete Decorators are classes that extend Decorator.

We can illustrate the decorator pattern with an example. We'll start by creating an Animal interface. The Animal interface is our component interface.

```

package za.ac.cput.kabasob.designpatterns.structural.decorator;

/**
 * @author boniface
 */
public interface Animal {

    public void describe();

}
  
```

LivingAnimal implements Animal and is our concrete component. Its `describe()` method displays a message indicating that it is an animal.

```

package za.ac.cput.kabasob.designpatterns.structural.decorator;
  
```

```
/**
 *
 * @author boniface
 */
public class LivingAnimal implements Animal {

    @Override
    public void describe() {
        System.out.println("\nI am an animal.");
    }
}
```

AnimalDecorator is our decorator abstract class. It implements Animal but since it is an abstract class, it does not have to implement describe(). Its constructor sets its Animal reference field.

```
package za.ac.cput.kabasob.designpatterns.structural.decorator;

/**
 *
 * @author boniface
 */
public abstract class AnimalDecorator implements Animal {

    Animal animal;

    public AnimalDecorator(Animal animal) {
        this.animal = animal;
    }
}
```

LegDecorator is a concrete decorator. Its constructor passes an Animal reference to AnimalDecorator's constructor. Its describe() method call's the Animal reference's describe() method and then outputs an additional message. It then calls its dance() method, showing that additional functionality can be added by the concrete decorator.

```
package za.ac.cput.kabasob.designpatterns.structural.decorator;

/**
 *
 * @author boniface
 */
public class LegDecorator extends AnimalDecorator {

    public LegDecorator(Animal animal) {
        super(animal);
    }

    @Override
    public void describe() {
        animal.describe();
        System.out.println("I have legs.");
        dance();
    }

    public void dance() {
        System.out.println("I can dance.");
    }
}
```

WingDecorator is a concrete decorator very similar to LegDecorator.

```
package za.ac.cput.kabasob.designpatterns.structural.decorator;

/**
 *
 * @author boniface
 */
public class WingDecorator extends AnimalDecorator {

    public WingDecorator(Animal animal) {
        super(animal);
    }

    @Override
    public void describe() {
        animal.describe();
    }
}
```

```

        System.out.println("I have wings.");
        fly();
    }

    public void fly() {
        System.out.println("I can fly.");
    }
}

```

GrowlDecorator is another concrete decorator.

```

package za.ac.cput.kabasob.designpatterns.structural.decorator;

/**
 *
 * @author boniface
 */
public class GrowlDecorator extends AnimalDecorator {

    public GrowlDecorator(Animal animal) {
        super(animal);
    }

    @Override
    public void describe() {
        animal.describe();
        growl();
    }

    public void growl() {
        System.out.println("Grrrrr.");
    }
}

```

The MainDriver class demonstrates the decorator design pattern. First, a LivingAnimal object is created and is referenced via the Animal reference. The describe() method is called. After this, we wrap the LivingAnimal in a LegDecorator object and once again call describe. We can see that legs have been added. After that, we wrap the LegDecorator in a WingDecorator, which adds wings to our animal. Later, we wrap the animal in two GrowlDecorators. From the describe() output, we can see that two growl messages are displayed since we wrapped the animal twice.

```

package za.ac.cput.kabasob.designpatterns.structural.decorator;

/**
 *
 * @author boniface
 */
public class MainDriver {

    public static void main(String[] args) {

        Animal animal = new LivingAnimal();
        animal.describe();

        animal = new LegDecorator(animal);
        animal.describe();

        animal = new WingDecorator(animal);
        animal.describe();

        animal = new GrowlDecorator(animal);
        animal = new GrowlDecorator(animal);
        animal.describe();
    }
}

```

Record your output in Lab Book

In this example, we just used the Animal reference to refer to the objects. Using this approach, we could only access the shared operations of the Animal interface (ie, the describe() method). Instead of referencing this interface, we could have referenced the concrete decorator class. This would have exposed the unique functionality of the concrete decorator.

7.7.3 Behavioral Design Patterns

7.7.4 Behavioral patterns

In software engineering, behavioral design patterns are design patterns that identify common communication patterns between objects and realize these patterns. By doing so, these patterns increase flexibility in carrying out this communication.

A behavioral pattern explains how objects interact. It describes how different objects and classes send messages to each other to make things happen and how the steps of a task are divided among different objects. It describes how different objects work together to accomplish a task

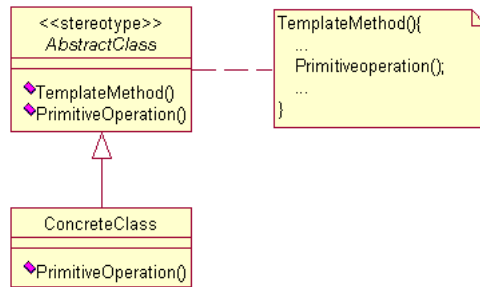
Where creational patterns mostly describe a moment of time (the instant of creation), and structural patterns describe a more or less static structure, behavioral patterns describe a process or a flow.

The interactions between cooperating objects should be such that they are communicating while maintaining as loose coupling as possible. The loose coupling is the key to n-tier architectures. In this, the implementation and the client should be loosely coupled in order to avoid hard-coding and dependencies.

Behavioral class patterns use inheritance, subclassing, and polymorphism to adjust the steps taken during a process. Behavioral class patterns focus on changing the exact algorithm used or task performed depending on circumstances.

1. Template Method Pattern
2. Mediator Pattern
3. Chain of Responsibility Pattern
4. Observer Pattern
5. Strategy Pattern
6. Command Pattern
7. State Pattern
8. Visitor Pattern
9. Iterator Pattern
10. Memento Pattern
11. Interpreter Pattern

7.7.4.1 Template Method Pattern



The template method pattern is a behavioral class pattern. A behavioral class pattern uses inheritance for distribution of behavior. In the template method pattern, a method (the 'template method') defines the steps of an algorithm. The implementation of these steps (ie, methods) can be deferred to subclasses. Thus, a particular algorithm is defined in the template method, but the exact steps of this algorithm can be defined in subclasses. The template method is implemented in an abstract class. The steps (methods) of the algorithm are declared in the abstract class, and the methods whose implementations are to be delegated to subclasses are declared abstract.

Here is an example of the template method pattern. Meal is an abstract class with a template method called doMeal() that defines the steps involved in a meal. We declare the method as final so that it can't be overridden. The algorithm defined by doMeal() consists of four steps: prepareIngredients(), cook(), eat(), and cleanUp(). The eat() method is implemented although subclasses can override the implementation. The prepareIngredients(), cook(), and cleanUp() methods are declared abstract so that subclasses need to implement them.

```

package za.ac.cput.kabasob.designpatterns.behavioral.templateMethod;

/**
 *
 * @author boniface
 */
public abstract class Meal {
    // template method

    public final void doMeal() {
        prepareIngredients();
        cook();
        eat();
        cleanUp();
    }

    public abstract void prepareIngredients();

    public abstract void cook();

    public void eat() {
        System.out.println("Mmm, that s good");
    }

    public abstract void cleanUp();
}
  
```

The HamburgerMeal class extends Meal and implements Meal's three abstract methods.

```

package za.ac.cput.kabasob.designpatterns.behavioral.templateMethod;

/**
 *
 * @author boniface
 */
public class HamburgerMeal extends Meal {
  
```



```

@Override
public void prepareIngredients() {
    System.out.println("Getting burgers, buns, and french fries");
}

@Override
public void cook() {
    System.out.println("Cooking burgers on grill and fries in oven");
}

@Override
public void cleanUp() {
    System.out.println("Throwing away paper plates");
}
}

```

The CHEeseBurgerMeal class implements Meal's three abstract methods and also overrides the eat() method.

```

package za.ac.cput.kabasob.designpatterns.behavioral.templateMethod;

/**
 *
 * @author boniface
 */
public class CheeseBurgerMeal extends Meal {
    @Override
    public void prepareIngredients() {
        System.out.println("Getting ground beef and Cheese");
    }

    @Override
    public void cook() {
        System.out.println("Cooking ground beef in pan");
    }

    @Override
    public void eat() {
        System.out.println("The Cheese Burgers are tasty");
    }

    @Override
    public void cleanUp() {
        System.out.println("Doing the dishes");
    }
}

```

The MainDriver creates a HamburgerMeal object and calls its doMeal() method. It creates a TacoMeal object and calls doMeal() on the TacoMeal object.

```

package za.ac.cput.kabasob.designpatterns.behavioral.templateMethod;

/**
 *
 * @author boniface
 */
public class MainDriver {

    public static void main(String[] args) {
        Meal meal1 = new HamburgerMeal();
        meal1.doMeal();

        System.out.println();

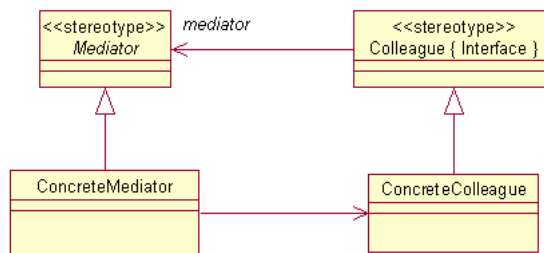
        Meal meal2 = new CheeseBurgerMeal();
        meal2.doMeal();
    }
}

```

Record your output in your log Book

As you can see, the template method design pattern allows us to define the steps in an algorithm and pass the implementation of these steps to subclasses.

7.7.4.2 Mediator Pattern



The mediator pattern is a behavioral object design pattern. The mediator pattern centralizes communication between objects into a mediator object. This centralization is useful since it localizes in one place the interactions between objects, which can increase code maintainability, especially as the number of classes in an application increases. Since communication occurs with the mediator rather than directly with other objects, the mediator pattern results in a loose coupling of objects.

The classes that communicate with the mediator are known as Colleagues. The mediator implementation is known as the Concrete Mediator. The mediator can have an interface that spells out the communication with Colleagues. Colleagues know their mediator, and the mediator knows its colleagues.

Now, let's look at an example of this pattern. We'll create a Mediator class (without implementing a mediator interface in this example). This mediator will mediate the communication between two buyers (a Swedish buyer and a French buyer), an American seller, and a currency converter.

The Mediator has references to the two buyers, the seller, and the converter. It has methods so that objects of these types can be registered. It also has a placeBid() method. This method takes a bid amount and a unit of currency as parameters. It converts this amount to a dollar amount via communication with the dollarConverter. It then asks the seller if the bid has been accepted, and it returns the answer.

```

package za.ac.cput.kabasob.designpatterns.behavioral.mediator;

/**
 *
 * @author boniface
 */
public class Mediator {

    Buyer swedishBuyer;
    Buyer frenchBuyer;
    AmericanSeller americanSeller;
    DollarConverter dollarConverter;

    public Mediator() {
    }

    public void registerSwedishBuyer(SwedishBuyer swedishBuyer) {
        this.swedishBuyer = swedishBuyer;
    }

    public void registerFrenchBuyer(FrenchBuyer frenchBuyer) {
        this.frenchBuyer = frenchBuyer;
    }

    public void registerAmericanSeller(AmericanSeller americanSeller) {
        this.americanSeller = americanSeller;
    }

    public void registerDollarConverter(DollarConverter dollarConverter) {
        this.dollarConverter = dollarConverter;
    }

    public boolean placeBid(float bid, String unitOfCurrency) {

```

```

        float dollarAmount = dollarConverter.convertCurrencyToDollars(bid, unitOfCurrency);
        return americanSeller.isBidAccepted(dollarAmount);
    }
}

```

Here is the Buyer class. The SwedishBuyer and FrenchBuyer classes are subclasses of Buyer. The buyer has a unit of currency as a field, and it also has a reference to the mediator. The Buyer class has a attemptToPurchase() method. This method submits a bid to the mediator's placeBid() method. It returns the mediator's response.

```

package za.ac.cput.kabasob.designpatterns.behavioral.mediator;

/**
 *
 * @author boniface
 */
class Buyer {

    Mediator mediator;
    String unitOfCurrency;

    public Buyer(Mediator mediator, String unitOfCurrency) {
        this.mediator = mediator;
        this.unitOfCurrency = unitOfCurrency;
    }

    public boolean attemptToPurchase(float bid) {
        System.out.println("Buyer attempting a bid of " + bid + " " + unitOfCurrency);
        return mediator.placeBid(bid, unitOfCurrency);
    }
}

```

The SwedishBuyer class is a subclass of Buyer. In the constructor, we set the unitOfCurrency to be "krona". We also register the SwedishBuyer with the mediator so that the mediator knows about the SwedishBuyer object.

```

package za.ac.cput.kabasob.designpatterns.behavioral.mediator;

/**
 *
 * @author boniface
 */
class SwedishBuyer extends Buyer {

    public SwedishBuyer(Mediator mediator) {
        super(mediator, "krona");
        this.mediator.registerSwedishBuyer(this);
    }
}

```

The FrenchBuyer class is similar to the SwedishBuyer class, except the unitOfCurrency is "euro", and it registers with the mediator as the FrenchBuyer.

```

package za.ac.cput.kabasob.designpatterns.behavioral.mediator;

/**
 *
 * @author boniface
 */
class FrenchBuyer extends Buyer {

    public FrenchBuyer(Mediator mediator) {
        super(mediator, "euro");
        this.mediator.registerFrenchBuyer(this);
    }
}

```

In the constructor of the AmericanSeller class, the class gets a reference to the mediator and the priceInDollars gets set. This is the price of some good being sold. The seller registers with the mediator as the AmericanSeller. The seller's isBidAccepted() method

takes a bid (in dollars). If the bid is over the price (in dollars), the bid is accepted and true is returned. Otherwise, false is returned.

```
package za.ac.cput.kabasob.designpatterns.behavioral.mediator;

/**
 *
 * @author boniface
 */
class AmericanSeller {
    Mediator mediator;
    float priceInDollars;

    public AmericanSeller(Mediator mediator, float priceInDollars) {
        this.mediator = mediator;
        this.priceInDollars = priceInDollars;
        this.mediator.registerAmericanSeller(this);
    }

    public boolean isBidAccepted(float bidInDollars) {
        if (bidInDollars >= priceInDollars) {
            System.out.println("Seller accepts the bid of " + bidInDollars + " dollars\n");
            return true;
        } else {
            System.out.println("Seller rejects the bid of " + bidInDollars + " dollars\n");
            return false;
        }
    }
}
```

The DollarConverter class is another colleague class. When created, it gets a reference to the mediator and registers itself with the mediator as the DollarConverter. This class has methods to convert amounts in euros and kronor to dollars.

```
package za.ac.cput.kabasob.designpatterns.behavioral.mediator;

/**
 *
 * @author boniface
 */
class DollarConverter {
    Mediator mediator;
    public static final float DOLLAR_UNIT = 1.0f;
    public static final float EURO_UNIT = 0.7f;
    public static final float KRONA_UNIT = 8.0f;

    public DollarConverter(Mediator mediator) {
        this.mediator = mediator;
        mediator.registerDollarConverter(this);
    }

    private float convertEurosToDollars(float euros) {
        float dollars = euros * (DOLLAR_UNIT / EURO_UNIT);
        System.out.println("Converting " + euros + " euros to " + dollars + " dollars");
        return dollars;
    }

    private float convertKronorToDollars(float kronor) {
        float dollars = kronor * (DOLLAR_UNIT / KRONA_UNIT);
        System.out.println("Converting " + kronor + " kronor to " + dollars + " dollars");
        return dollars;
    }

    public float convertCurrencyToDollars(float amount, String unitOfCurrency) {
        if ("krona".equalsIgnoreCase(unitOfCurrency)) {
            return convertKronorToDollars(amount);
        } else {
            return convertEurosToDollars(amount);
        }
    }
}
```

The MainDriver demonstrates our mediator pattern. It creates a SwedishBuyer object and a FrenchBuyer object. It creates an AmericanSeller object with a selling price set to 10 dollars. It then creates a DollarConverter. All of these objects register themselves with the mediator in their constructors. The Swedish buyer starts with a bid of 55 kronor and keeps bidding up in increments of 15 kronor until the bid is accepted. The

French buyer starts bidding at 3 euros and keeps bidding in increments of 1.50 euros until the bid is accepted.

```
package za.ac.cput.kabasob.designpatterns.behavioral.mediator;

/**
 *
 * @author boniface
 */
public class MainDriver {
    public static void main(String[] args) {
        Mediator mediator = new Mediator();

        Buyer swedishBuyer = new SwedishBuyer(mediator);
        Buyer frenchBuyer = new FrenchBuyer(mediator);
        float sellingPriceInDollars = 10.0f;
        AmericanSeller americanSeller = new AmericanSeller(mediator, sellingPriceInDollars);
        DollarConverter dollarConverter = new DollarConverter(mediator);

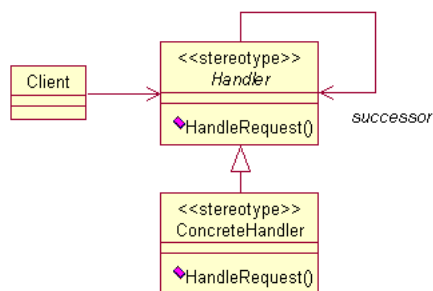
        float swedishBidInKronor = 55.0f;
        while (!swedishBuyer.attemptToPurchase(swedishBidInKronor)) {
            swedishBidInKronor += 15.0f;
        }

        float frenchBidInEuros = 3.0f;
        while (!frenchBuyer.attemptToPurchase(frenchBidInEuros)) {
            frenchBidInEuros += 1.5f;
        }
    }
}
```

Record your output in the log Book

In this example of the mediator pattern, notice that all communication between our objects (buyers, seller, and converter) occurs via the mediator. The mediator pattern helps reduce the number of object references needed (via composition) as classes proliferate in a project as a project grows.

7.7.4.3 Chain of Responsibility Pattern



The chain of responsibility pattern is a behavioral object design pattern. In the chain of responsibility pattern, a series of handler objects are chained together to handle a request made by a client object. If the first handler can't handle the request, the request is forwarded to the next handler, and it is passed down the chain until the request reaches a handler that can handle the request or the chain ends. In this pattern, the client is decoupled from the actual handling of the request, since it does not know what class will actually handle the request.

In this pattern, a Handler is an interface for handling a request and accessing a handler's successor. A Handler is implemented by a Concrete Handler. The Concrete Handler will handle the request or pass it on to the next Concrete Handler. A Client makes the request to the start of the handler chain.

Now, let's look at an example of the chain of responsibility pattern. Rather than an interface, We'll use an abstract base class as the handler so that subclasses can utilize the implemented `setSuccessor()` method. This abstract class is called `PlanetHandler`. Concrete handlers that subclass `PlanetHandler` need to implement the `handleRequest()` method.

```
package za.ac.cput.kabasob.designpatterns.behavioral.chainofresponsibility;

/**
 *
 * @author boniface
 */
public abstract class PlanetHandler {

    PlanetHandler successor;

    public void setSuccessor(PlanetHandler successor) {
        this.successor = successor;
    }

    public abstract void handleRequest(PlanetEnum request);
}
```

This example will utilize an enum of the planets called `PlanetEnum`.

```
package za.ac.cput.kabasob.designpatterns.behavioral.chainofresponsibility;

/**
 *
 * @author boniface
 */
public enum PlanetEnum {

    MERCURY, VENUS, EARTH, MARS, JUPITER, SATURN, URANUS, NEPTUNE;
}
```

`MercuryHandler` subclasses `PlanetHandler` and implements the `handleRequest()` method. If the request is a `PlanetEnum.MERCURY`, it will handle the request. Otherwise, the request is passed on to this handler's successor if the successor exists.

```
package za.ac.cput.kabasob.designpatterns.behavioral.chainofresponsibility;

/**
 *
 * @author boniface
 */
public class MercuryHandler extends PlanetHandler {

    public void handleRequest(PlanetEnum request) {
        if (request == PlanetEnum.MERCURY) {
            System.out.println("MercuryHandler handles " + request);
            System.out.println("Mercury is hot.\n");
        } else {
            System.out.println("MercuryHandler doesn't handle " + request);
            if (successor != null) {
                successor.handleRequest(request);
            }
        }
    }
}
```

`VenusHandler` is similar to `MercuryHandler`, except it handles `PlanetEnum.VENUS` requests.

```
package za.ac.cput.kabasob.designpatterns.behavioral.chainofresponsibility;

/**
 *
 * @author boniface
 */
public class VenusHandler extends PlanetHandler {

    public void handleRequest(PlanetEnum request) {
        if (request == PlanetEnum.VENUS) {
```

```

        System.out.println("VenusHandler handles " + request);
        System.out.println("Venus is poisonous.\n");
    } else {
        System.out.println("VenusHandler doesn't handle " + request);
        if (successor != null) {
            successor.handleRequest(request);
        }
    }
}
}
}

```

EarthHandler similarly handles PlanetEnum.EARTH requests.

```

package za.ac.cput.kabasob.designpatterns.behavioral.chainofresponsibility;

/**
 * @author boniface
 */
public class EarthHandler extends PlanetHandler {

    public void handleRequest(PlanetEnum request) {
        if (request == PlanetEnum.EARTH) {
            System.out.println("EarthHandler handles " + request);
            System.out.println("Earth is comfortable.\n");
        } else {
            System.out.println("EarthHandler doesn't handle " + request);
            if (successor != null) {
                successor.handleRequest(request);
            }
        }
    }
}

```

The MainDriver is the client class. It creates the chain of handlers, starting with MercuryHandler, then VenusHandler, and then EarthHandler. The setUpChain() method returns the chain to main() via a PlanetHandler reference. Four requests are made of the chain, where the requests are VENUS, MERCURY, EARTH, and JUPITER.

```

package za.ac.cput.kabasob.designpatterns.behavioral.chainofresponsibility;

/**
 * @author boniface
 */
public class MainDriver {

    public static void main(String[] args) {
        PlanetHandler chain = setUpChain();
        //Submit Requests
        chain.handleRequest(PlanetEnum.VENUS);
        chain.handleRequest(PlanetEnum.MERCURY);
        chain.handleRequest(PlanetEnum.EARTH);
        chain.handleRequest(PlanetEnum.JUPITER);
    }

    public static PlanetHandler setUpChain() {
        PlanetHandler mercuryHandler = new MercuryHandler();
        PlanetHandler venusHandler = new VenusHandler();
        PlanetHandler earthHandler = new EarthHandler();

        mercuryHandler.setSuccessor(venusHandler);
        venusHandler.setSuccessor(earthHandler);

        return mercuryHandler;
    }
}

```

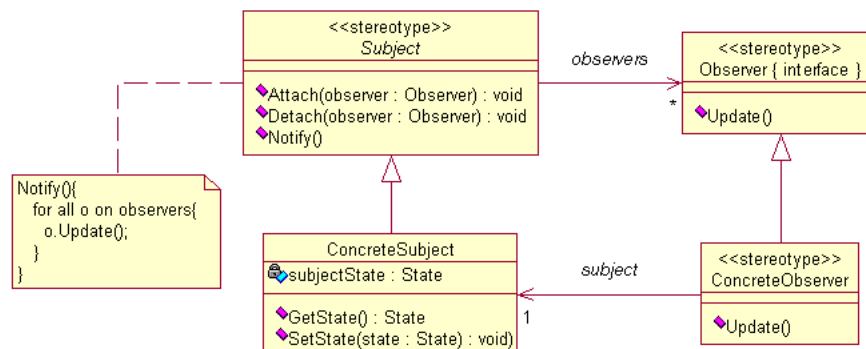
Record the output in your lab book

Notice that if a handler can't handle the request, it passes the request on to the next handler.

Notice that the last request made of the chain is JUPITER. This request is not handled by any handlers, demonstrating that a request does not have to be handled by any

handlers. If we had wanted to, we could have also written an OtherPlanets handler and placed it at the end of the chain to handle planet requests not handled by other previous handlers. This would demonstrate that we can make our handlers specific at the beginning of the chain and more general at the end of the chain, thus handling broader categories of requests as we approach the end of the chain.

7.7.4.4 Observer Pattern



The observer pattern is a behavioral object design pattern. In the observer pattern, an object called the subject maintains a collection of objects called observers. When the subject changes, it notifies the observers. Observers can be added or removed from the collection of observers in the subject. The changes in state of the subject can be passed to the observers so that the observers can change their own state to reflect this change.

The subject has an interface that defines methods for attaching and detaching observers from the subject's collection of observers. This interface also features a notification method. This method should be called when the state of the subject changes. This notifies the observers that the subject's state has changed. The observers have an interface with a method to update the observer. This update method is called for each observer in the subject's notification method. Since this communication occurs via an interface, any concrete observer implementing the observer interface can be updated by the subject. This results in loose coupling between the subject and the observer classes.

Now we'll look at an example of the observer pattern. We'll start by creating an interface for the subject called WeatherSubject. This will declare three methods: addObserver(), removeObserver(), and doNotify().

```

package za.ac.cput.kabasob.designpatterns.behavioral.observer;

/**
 *
 * @author boniface
 */
public interface WeatherSubject {

    public void addObserver(WeatherObserver weatherObserver);

    public void removeObserver(WeatherObserver weatherObserver);

    public void doNotify();
}

```

We'll also create an interface for the observers called WeatherObserver. It features one method, a doUpdate() method.


```

package za.ac.cput.kabasob.designpatterns.behavioral.observer;

/**
 *
 * @author boniface
 */
public interface WeatherObserver {

    public void doUpdate(int temperature);

}

```

The WeatherStation class implements WeatherSubject. It is our subject class. It maintains a set of WeatherObservers which are added via addObserver() and removed via removeObserver(). When WeatherSubject's state changes via setTemperature(), the doNotify() method is called, which contacts all the WeatherObservers with the temperature via their doUpdate() methods.

```

package za.ac.cput.kabasob.designpatterns.behavioral.observer;

import java.util.HashSet;
import java.util.Iterator;
import java.util.Set;

/**
 *
 * @author boniface
 */
public class WeatherStation implements WeatherSubject {

    Set<WeatherObserver> weatherObservers;
    int temperature;

    public WeatherStation(int temperature) {
        weatherObservers = new HashSet<WeatherObserver>();
        this.temperature = temperature;
    }

    @Override
    public void addObserver(WeatherObserver weatherObserver) {
        weatherObservers.add(weatherObserver);
    }

    @Override
    public void removeObserver(WeatherObserver weatherObserver) {
        weatherObservers.remove(weatherObserver);
    }

    @Override
    public void doNotify() {
        Iterator<WeatherObserver> it = weatherObservers.iterator();
        while (it.hasNext()) {
            WeatherObserver weatherObserver = it.next();
            weatherObserver.doUpdate(temperature);
        }
    }

    public void setTemperature(int newTemperature) {
        System.out.println("\nWeather station setting temperature to " + newTemperature);
        temperature = newTemperature;
        doNotify();
    }

}

```

WeatherCustomer1 is an observer that implements WeatherObserver. Its doUpdate() method gets the current temperature from the WeatherStation and displays it.

```

package za.ac.cput.kabasob.designpatterns.behavioral.observer;

/**
 *
 * @author boniface
 */
public class WeatherCustomer1 implements WeatherObserver {

    @Override
    public void doUpdate(int temperature) {
        System.out.println("Weather customer 1 just found out the temperature is:" + temperature);
    }

}

```

WeatherCustomer2 performs similar functionality as WeatherCustomer1.

```
package za.ac.cput.kabasob.designpatterns.behavioral.observer;

/**
 *
 * @author boniface
 */
public class WeatherCustomer2 implements WeatherObserver {

    @Override
    public void doUpdate(int temperature) {
        System.out.println("Weather customer 2 just found out the temperature is:" + ↵
            temperature);
    }
}
```

The MainDriver class demonstrates the observer pattern. It creates a WeatherStation and then a WeatherCustomer1 and a WeatherCustomer2. The two customers are added as observers to the weather station. Then the setTemperature() method of the weather station is called. This changes the state of the weather station and the customers are notified of this temperature update. Next, the WeatherCustomer1 object is removed from the station's collection of observers. Then, the setTemperature() method is called again. This results in the notification of the WeatherCustomer2 object.

```
package za.ac.cput.kabasob.designpatterns.behavioral.observer;

/**
 *
 * @author boniface
 */
public class MainDriver {

    public static void main(String[] args) {
        WeatherStation weatherStation = new WeatherStation(33);

        WeatherCustomer1 wc1 = new WeatherCustomer1();
        WeatherCustomer2 wc2 = new WeatherCustomer2();
        weatherStation.addObserver(wc1);
        weatherStation.addObserver(wc2);

        weatherStation.setTemperature(34);

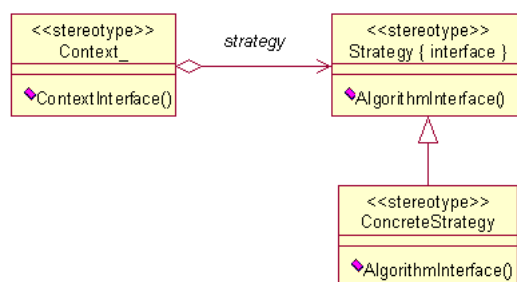
        weatherStation.removeObserver(wc1);

        weatherStation.setTemperature(35);
    }
}
```

Record output in your log book

In a more advanced case, we might have given each observer a reference to the weather station object. This could allow the observer the ability to compare the state of the subject in detail with its own state and make any necessary updates to its own state.

7.7.4.5 Strategy Pattern



The strategy pattern is a behavioral object design pattern. In the strategy pattern, different algorithms are represented as Concrete Strategy classes, and they share a common Strategy interface. A Context object contains a reference to a Strategy. By changing the Context's Strategy, different behaviors can be obtained. Although these behaviors are different, the different strategies all operate on data from the Context.

The strategy pattern is one way that composition can be used as an alternative to subclassing. Rather than providing different behaviors via subclasses overriding methods in superclasses, the strategy pattern allows different behaviors to be placed in Concrete Strategy classes which share the common Strategy interface. A Context class is composed of a reference to a Strategy.

Here is an example of the strategy pattern. First, we'll define a Strategy interface. It declares a `checkTemperature()` method.

```
package za.ac.cput.kabasob.designpatterns.behavioral.strategy;

/**
 *
 * @author boniface
 */
public interface Strategy {
    boolean checkTemperature(int temperatureInF);
}
```

The `HikeStrategy` class is a concrete strategy class that implements the Strategy interface. The `checkTemperature` method is implemented so that if the temperature is between 50 and 90, it returns true. Otherwise it returns false.

```
package za.ac.cput.kabasob.designpatterns.behavioral.strategy;

/**
 *
 * @author boniface
 */
public class HikeStrategy implements Strategy {

    @Override
    public boolean checkTemperature(int temperatureInF) {
        if ((temperatureInF >= 50) && (temperatureInF <= 90)) {
            return true;
        } else {
            return false;
        }
    }
}
```

The `SkiStrategy` implements the Strategy interface. If the temperature is 32 or less, the `checkTemperature` method returns true. Otherwise it returns false.

```
package za.ac.cput.kabasob.designpatterns.behavioral.strategy;

/**
 *
 * @author boniface
 */
public class SkiStrategy implements Strategy {

    @Override
    public boolean checkTemperature(int temperatureInF) {
        if (temperatureInF <= 32) {
            return true;
        } else {
            return false;
        }
    }
}
```

The Context class contains a temperature and a reference to a Strategy. The Strategy can be changed, resulting in different behavior that operates on the same data in the Context. The result of this can be obtained from the Context via the getResult() method.

```
package za.ac.cput.kabasob.designpatterns.behavioral.strategy;

/**
 *
 * @author boniface
 */
public class Context {

    int temperatureInF;
    Strategy strategy;

    public Context(int temperatureInF, Strategy strategy) {
        this.temperatureInF = temperatureInF;
        this.strategy = strategy;
    }

    public void setStrategy(Strategy strategy) {
        this.strategy = strategy;
    }

    public int getTemperatureInF() {
        return temperatureInF;
    }

    public boolean getResult() {
        return strategy.checkTemperature(temperatureInF);
    }
}
```

The MainDriver class creates a Context object with a temperature of 60 and with a SkiStrategy. It displays the temperature from the context and whether that temperature is OK for skiing. After that, it sets the Strategy in the Context to HikeStrategy. It then displays the temperature from the context and whether that temperature is OK for hiking.

```
package za.ac.cput.kabasob.designpatterns.behavioral.strategy;

/**
 *
 * @author boniface
 */
public class MainDriver {

    public static void main(String[] args) {

        int temperatureInF = 60;

        Strategy skiStrategy = new SkiStrategy();
        Context context = new Context(temperatureInF, skiStrategy);

        System.out.println("Is the temperature (" + context.getTemperatureInF() + "F) good ←  
for skiing? " +  
context.getResult());

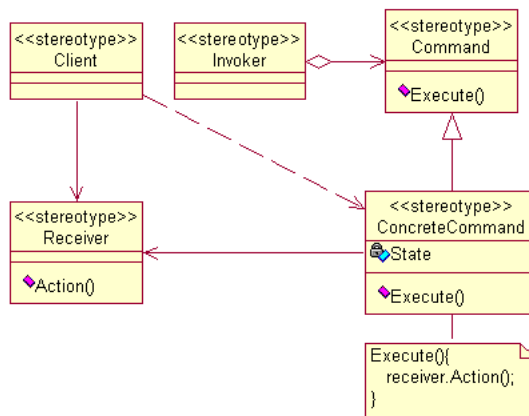
        Strategy hikeStrategy = new HikeStrategy();
        context.setStrategy(hikeStrategy);

        System.out.println("Is the temperature (" + context.getTemperatureInF() + "F) good ←  
for hiking? " +  
context.getResult());

    }
}
```

Record output in your log book

7.7.4.6 Command Pattern



The command pattern is a behavioral object design pattern. In the command pattern, a Command interface declares a method for executing a particular action. Concrete Command classes implement the execute() method of the Command interface, and this execute() method invokes the appropriate action method of a Receiver class that the Concrete Command class contains. The Receiver class performs a particular action. A Client class is responsible for creating a Concrete Command and setting the Receiver of the Concrete Command. An Invoker class contains a reference to a Command and has a method to execute the Command.

In the command pattern, the invoker is decoupled from the action performed by the receiver. The invoker has no knowledge of the receiver. The invoker invokes a command, and the command executes the appropriate action of the receiver. Thus, the invoker can invoke commands without knowing the details of the action to be performed. In addition, this decoupling means that changes to the receiver's action don't directly affect the invocation of the action.

The command pattern can be used to perform 'undo' functionality. In this case, the Command interface should include an unexecute() method.

Here is an example of the command pattern. We have a Command interface with an execute() method.

```

package za.ac.cput.kabasob.designpatterns.behavioral.command;

/**
 *
 * @author boniface
 */
public interface Command {

    public void execute();

}
  
```

LunchCommand implements Command. It contains a reference to Lunch, a receiver. Its execute() method invokes the appropriate action on the receiver.

```

package za.ac.cput.kabasob.designpatterns.behavioral.command;

/**
 *
 * @author boniface
 */
  
```

```
public class LunchCommand implements Command {  
    Lunch lunch;  
  
    public LunchCommand(Lunch lunch) {  
        this.lunch = lunch;  
    }  
  
    @Override  
    public void execute() {  
        lunch.makeLunch();  
    }  
}
```

The DinnerCommand is similar to LunchCommand. It contains a reference to Dinner, a receiver. Its execute() method invokes the makeDinner() action of the Dinner object.

```
package za.ac.cput.kabasob.designpatterns.behavioral.command;  
  
/**  
 *  
 * @author boniface  
 */  
public class DinnerCommand implements Command {  
    Dinner dinner;  
  
    public DinnerCommand(Dinner dinner) {  
        this.dinner = dinner;  
    }  
  
    @Override  
    public void execute() {  
        dinner.makeDinner();  
    }  
}
```

Lunch is a receiver.

```
package za.ac.cput.kabasob.designpatterns.behavioral.command;  
  
/**  
 *  
 * @author boniface  
 */  
public class Lunch {  
    public void makeLunch() {  
        System.out.println("Lunch is being made");  
    }  
}
```

Dinner is also a receiver.

```
package za.ac.cput.kabasob.designpatterns.behavioral.command;  
  
/**  
 *  
 * @author boniface  
 */  
public class Dinner {  
    public void makeDinner() {  
        System.out.println("Dinner is being made");  
    }  
}
```

MealInvoker is the invoker class. It contains a reference to the Command to invoke. Its invoke() method calls the execute() method of the Command.

```
package za.ac.cput.kabasob.designpatterns.behavioral.command;  
  
/**  
 *  
 * @author boniface
```

```

*/
public class MealInvoker {

    Command command;

    public MealInvoker(Command command) {
        this.command = command;
    }

    public void setCommand(Command command) {
        this.command = command;
    }

    public void invoke() {
        command.execute();
    }
}

```

The MainDriver class demonstrates the command pattern. It instantiates a Lunch (receiver) object and creates a LunchCommand (concrete command) with the Lunch. The LunchCommand is referenced by a Command interface reference. Next, we perform the same procedure on the Dinner and DinnerCommand objects. After this, we create a MealInvoker object with lunchCommand, and we call the invoke() method of mealInvoker. After this, we set mealInvoker's command to dinnerCommand, and once again call invoke() on mealInvoker.

```

package za.ac.cput.kabasob.designpatterns.behavioral.command;

/**
 *
 * @author boniface
 */
public class MainDriver {
    public static void main(String[] args) {

        Lunch lunch = new Lunch(); // receiver
        Command lunchCommand = new LunchCommand(lunch); // concrete command

        Dinner dinner = new Dinner(); // receiver
        Command dinnerCommand = new DinnerCommand(dinner); // concrete command

        MealInvoker mealInvoker = new MealInvoker(lunchCommand); // invoker
        mealInvoker.invoke();

        mealInvoker.setCommand(dinnerCommand);
        mealInvoker.invoke();

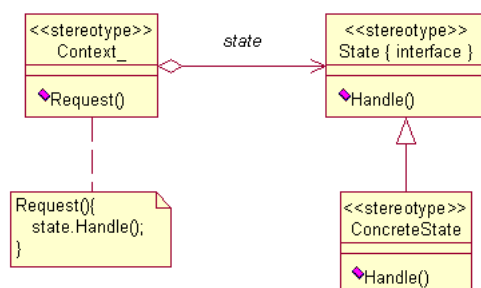
    }
}

```

Record output in your log book

As you can see, the invoker invokes a command, but has no direct knowledge of the action being performed by the receiver.

7.7.4.7 State Pattern



The state pattern is a behavioral object design pattern. The idea behind the state pattern is for an object to change its behavior depending on its state. In the state pattern, we have a Context class, and this class has a State reference to a Concrete State instance. The State interface declares particular methods that represent the behaviors of a particular state. Concrete States implement these behaviors. By changing a Context's Concrete State, we change its behavior. In essence, in the state pattern, a class (the Context) is supposed to behave like different classes depending on its state. The state pattern avoids the use of switch and if statements to change behavior.

Let's look at an example of the state pattern. First off, we'll define the EmotionalState interface. It declares two methods, sayHello() and sayGoodbye().

```
package za.ac.cput.kabasob.designpatterns.behavioral.state;

/**
 *
 * @author boniface
 */
public interface EmotionalState {

    public String sayHello();

    public String sayGoodbye();

}
```

The HappyState class is a Concrete State that implements sayHello() and sayGoodbye() of EmotionalState. These messages are cheerful (representing a happy state).

```
package za.ac.cput.kabasob.designpatterns.behavioral.state;

/**
 *
 * @author boniface
 */
// Concrete State
public class HappyState implements EmotionalState {

    @Override
    public String sayGoodbye() {
        return "Bye, friend!";
    }

    @Override
    public String sayHello() {
        return "Hello, friend!";
    }

}
```

The SadState class also implements the EmotionalState interface. The messages are sad (representing a sad state).

```
package za.ac.cput.kabasob.designpatterns.behavioral.state;

/**
 *
 * @author boniface
 */
// Concrete State
public class SadState implements EmotionalState {

    @Override
    public String sayGoodbye() {
        return "Bye. Sniff, sniff.";
    }

    @Override
    public String sayHello() {
        return "Hello. Sniff, sniff.";
    }

}
```


The Person class is the Context class. It contains an EmotionalState reference to a concrete state. In this example, we have Person implement the EmotionalState reference, and we pass the calls to Person's sayHello() and sayGoodbye() methods on to the corresponding methods on the emotionalState reference. As a result of this, a Person object behaves differently depending on the state of Person (ie, the current EmotionalState reference).

```
package za.ac.cput.kabasob.designpatterns.behavioral.state;

/**
 *
 * @author boniface
 */
// Context
public class Person implements EmotionalState {

    EmotionalState emotionalState;

    public Person(EmotionalState emotionalState) {
        this.emotionalState = emotionalState;
    }

    public void setEmotionalState(EmotionalState emotionalState) {
        this.emotionalState = emotionalState;
    }

    @Override
    public String sayGoodbye() {
        return emotionalState.sayGoodbye();
    }

    @Override
    public String sayHello() {
        return emotionalState.sayHello();
    }
}
```

The MainDriver class demonstrates the state pattern. First, it creates a Person object with a HappyState object. We display the results of sayHello() and sayGoodbye() when the person object is in the happy state. Next, we change the person object's state with a SadState object. We display the results of sayHello() and sayGoodbye(), and we see that in the sad state, the person object's behavior is different.

```
package za.ac.cput.kabasob.designpatterns.behavioral.state;

/**
 *
 * @author boniface
 */
public class MainDriver {
    public static void main(String[] args) {

        Person person = new Person(new HappyState());
        System.out.println("Hello in happy state: " + person.sayHello());
        System.out.println("Goodbye in happy state: " + person.sayGoodbye());

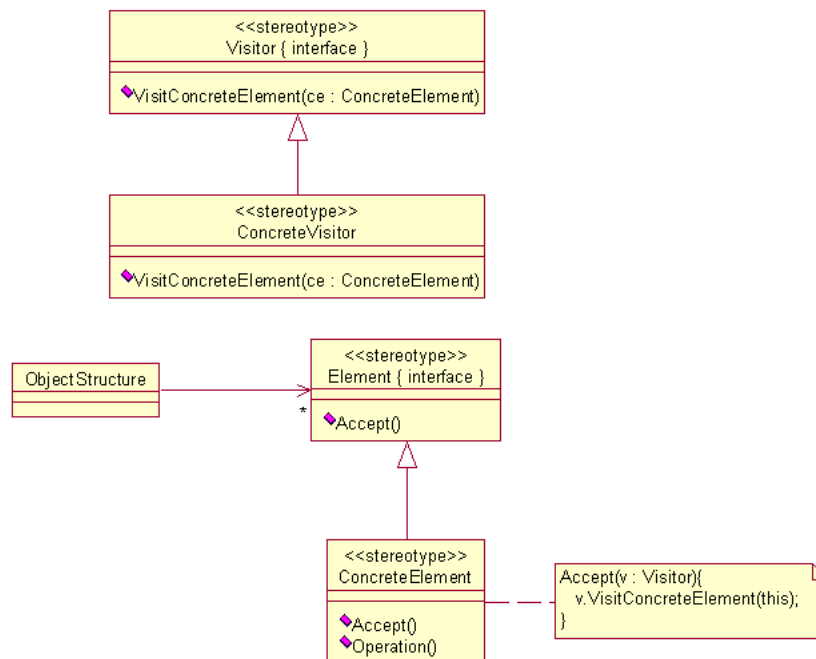
        person.setEmotionalState(new SadState());
        System.out.println("Hello in sad state: " + person.sayHello());
        System.out.println("Goodbye in sad state: " + person.sayGoodbye());

    }
}
```

Record output in your log book

Note that we don't necessarily need to have the Context (ie, Person) implement the EmotionalState interface. The behavioral changes could have been internal to the Context rather than exposing EmotionalState's methods to the outside. However, having the Context class implement the State interface allows us to directly access the different behaviors that result from the different states of the Context.

7.7.4.8 Visitor Pattern



The visitor pattern is a behavioral object design pattern. The visitor pattern is used to simplify operations on groupings of related objects. These operations are performed by the visitor rather than by placing this code in the classes being visited. Since the operations are performed by the visitor rather than by the classes being visited, the operation code gets centralized in the visitor rather than being spread out across the grouping of objects, thus leading to code maintainability. The visitor pattern also avoids the use of the `instanceof` operator in order to perform calculations on similar classes.

In the visitor pattern, we have a **Visitor** interface that declares `visit()` methods for the various types of elements that can be visited. Concrete Visitors implement the **Visitor** interface's `visit()` methods. The `visit()` methods are the operations that should be performed by the visitor on an element being visited.

The related classes that will be visited implement an **Element** interface that declares an `accept()` method that takes a visitor as an argument. Concrete Elements implement the **Element** interface and implement the `accept()` method. In the `accept()` method, the visitor's `visit()` method is called with 'this', the current object of the Concrete Element type.

The elements to visit all implement the `accept()` method that takes a visitor as an argument. In this method, they call the visitor's `visit()` method with 'this'. As a result of this, an element takes a visitor and then the visitor performs its operation on the element.

Let's illustrate the visitor pattern with an example. First, we'll define a **NumberVisitor** interface. This interface declares three visit methods which take different types as arguments. Note that if we only wrote one visit method, we'd have to use the `instanceof` operator or a similar technique to handle the different element types. However, since

we have separate visit methods, we don't need the instanceof operator, since each visit method handles a different type.

```
package za.ac.cput.kabasob.designpatterns.behavioral.visitor;

import java.util.List;

/**
 *
 * @author boniface
 */
public interface NumberVisitor {

    public void visit(TwoElement twoElement);

    public void visit(ThreeElement threeElement);

    public void visit(List<NumberElement> elementList);

}
```

All of the elements classes to be visited will implement the NumberElement interface. This interface has a single method that takes a NumberVisitor as an argument

```
package za.ac.cput.kabasob.designpatterns.behavioral.visitor;

/**
 *
 * @author boniface
 */
public interface NumberElement {

    public void accept(NumberVisitor visitor);

}
```

Let's create a TwoElement class that implements NumberElement. It has two int fields. Its accept() method calls the visitor's visit() method with 'this'. The operator to be performed on TwoElement is performed by the visitor.

```
package za.ac.cput.kabasob.designpatterns.behavioral.visitor;

/**
 *
 * @author boniface
 */
public class TwoElement implements NumberElement{

    int a;
    int b;

    public TwoElement(int a, int b) {
        this.a = a;
        this.b = b;
    }

    @Override
    public void accept(NumberVisitor visitor) {
        visitor.visit(this);
    }

}
```

The ThreeElement class is similar to TwoElement, except that it has three int fields.

```
package za.ac.cput.kabasob.designpatterns.behavioral.visitor;

/**
 *
 * @author boniface
 */
public class ThreeElement implements NumberElement {

    int a;
    int b;
    int c;

    public ThreeElement(int a, int b, int c) {
        this.a = a;
        this.b = b;
    }

}
```

```

        this.c = c;
    }

    @Override
    public void accept(NumberVisitor visitor) {
        visitor.visit(this);
    }
}

```

Now, let's create a visitor called SumVisitor that implements the NumberVisitor interface. For TwoElement and ThreeElement objects, this visitor will sum up the int fields. For a List of NumElements (ie, TwoElement and ThreeElement objects), this visitor will iterate over the elements and call their accept() methods. As a result of this, the visitor will perform visit operations on all the TwoElement and ThreeElement objects that make up the list, since the call to accept() in turn calls the visitor's visit methods for the TwoElement and ThreeElement objects.

```

package za.ac.cput.kabasob.designpatterns.behavioral.visitor;

import java.util.List;

/**
 *
 * @author boniface
 */
public class SumVisitor implements NumberVisitor {

    @Override
    public void visit(TwoElement twoElement) {
        int sum = twoElement.a + twoElement.b;
        System.out.println(twoElement.a + "+" + twoElement.b + "=" + sum);
    }

    @Override
    public void visit(ThreeElement threeElement) {
        int sum = threeElement.a + threeElement.b + threeElement.c;
        System.out.println(threeElement.a + "+" + threeElement.b + "+" + threeElement.c + "=" + sum);
    }

    @Override
    public void visit(List<NumberElement> elementList) {
        for (NumberElement ne : elementList) {
            ne.accept(this);
        }
    }
}

```

Here is another visitor, TotalSumVisitor. In addition to summing up the int fields and displaying the sum, this visitor will keep track of the total sums of all the elements that are visited.

```

package za.ac.cput.kabasob.designpatterns.behavioral.visitor;

import java.util.List;

/**
 *
 * @author boniface
 */
public class TotalSumVisitor implements NumberVisitor {

    int totalSum = 0;

    @Override
    public void visit(TwoElement twoElement) {
        int sum = twoElement.a + twoElement.b;
        System.out.println("Adding " + twoElement.a + "+" + twoElement.b + "=" + sum + " to total");
        totalSum += sum;
    }

    @Override
    public void visit(ThreeElement threeElement) {
        int sum = threeElement.a + threeElement.b + threeElement.c;
        System.out.println("Adding " + threeElement.a + "+" + threeElement.b + "+" + threeElement.c + "=" + sum + " to total");
    }
}

```

```

        totalSum += sum;
    }

    @Override
    public void visit(List<NumberElement> elementList) {
        for (NumberElement ne : elementList) {
            ne.accept(this);
        }
    }

    public int getTotalSum() {
        return totalSum;
    }
}

```

Let's see the visitor pattern in action. The MainDriver class creates two TwoElement objects and one ThreeElement object. It creates a list of NumberElements and adds the TwoElement object and the ThreeElement object to the list. Next, we create a SumVisitor and we visit the list with the SumVisitor. After this, we create a TotalSumVisitor and visit the list with the TotalSumVisitor. We display the total sum via the call to TotalSumVisitor's getTotalSum() method.

```

package za.ac.cput.kabasob.designpatterns.behavioral.visitor;

import java.util.ArrayList;
import java.util.List;

/**
 *
 * @author boniface
 */
public class MainDriver {
    public static void main(String[] args) {

        TwoElement two1 = new TwoElement(3, 3);
        TwoElement two2 = new TwoElement(2, 7);
        ThreeElement three1 = new ThreeElement(3, 4, 5);

        List<NumberElement> numberElements = new ArrayList<NumberElement>();
        numberElements.add(two1);
        numberElements.add(two2);
        numberElements.add(three1);

        System.out.println("Visiting element list with SumVisitor");
        NumberVisitor sumVisitor = new SumVisitor();
        sumVisitor.visit(numberElements);

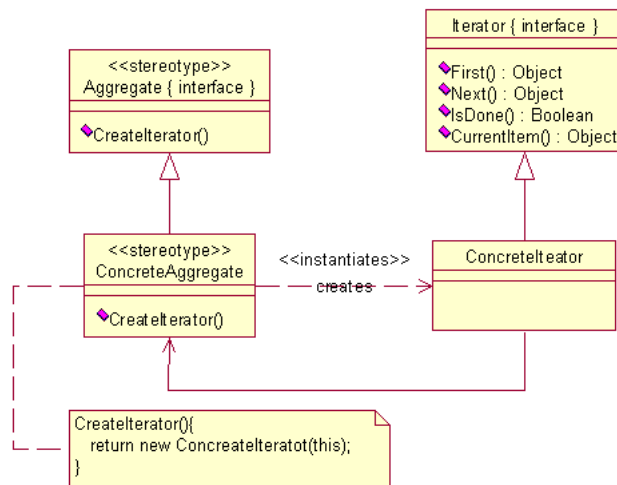
        System.out.println("\nVisiting element list with TotalSumVisitor");
        TotalSumVisitor totalSumVisitor = new TotalSumVisitor();
        totalSumVisitor.visit(numberElements);
        System.out.println("Total sum:" + totalSumVisitor.getTotalSum());
    }
}

```

Record output in your log book

Notice that if we'd like to perform new operations on the grouping of elements, all we would need to do is write a new visitor class. We would not have to make any additions to the existing element classes, since they provide the data but none of the code for the operations.

7.7.4.9 Iterator Pattern



The iterator pattern is a behavioral object design pattern. The iterator pattern allows for the traversal through the elements in a grouping of objects via a standardized interface. An Iterator interface defines the actions that can be performed. These actions include being able to traverse the objects and also obtain the objects.

Java features the widely used `java.util.Iterator` interface which is used to iterate through things such as Java collections. We can write our own iterator by implementing `java.util.Iterator`. This interface features the `hasNext()`, `next()`, and `remove()` methods. When writing an iterator for a class, it is very common for the iterator class to be an inner class of the class that we'd like to iterate through.

Let's look at an example of this. We have an `Item` class, which represents an item on a menu. An item has a name and a price.

```

package za.ac.cput.kabasob.designpatterns.behavioral.iterator;

/**
 *
 * @author boniface
 */
public class Item {
    String name;
    float price;

    public Item(String name, float price) {
        this.name = name;
        this.price = price;
    }

    public String toString() {
        return name + ": \\\$" + price;
    }
}
  
```

Here is the `Menu` class. It has a list of menu items of type `Item`. Items can be added via the `addItem()` method. The `iterator()` method returns an iterator of menu items. The `MenuItem` class is an inner class of `Menu` that implements the `Iterator` interface for `Item` objects. It contains basic implementations of the `hasNext()`, `next()`, and `remove()` methods.

```

package za.ac.cput.kabasob.designpatterns.behavioral.iterator;
  
```

```

import java.util.ArrayList;
import java.util.Iterator;
import java.util.List;

/**
 *
 * @author boniface
 */
public class Menu {

    List<Item> menuItems;

    public Menu() {
        menuItems = new ArrayList<Item>();
    }

    public void addItem(Item item) {
        menuItems.add(item);
    }

    public Iterator<Item> iterator() {
        return new MenuItemIterator();
    }

    class MenuItemIterator implements Iterator<Item> {
        int currentIndex = 0;

        @Override
        public boolean hasNext() {
            if (currentIndex >= menuItems.size()) {
                return false;
            } else {
                return true;
            }
        }

        @Override
        public Item next() {
            return menuItems.get(currentIndex++);
        }

        @Override
        public void remove() {
            menuItems.remove(--currentIndex);
        }
    }
}

```

The MainDriver class demonstrates the iterator pattern. It creates three items and adds them to the menu object. Next, it gets an Item iterator from the menu object and iterates over the items in the menu. After this, it calls remove() to remove the last item obtained by the iterator. Following this, it gets a new iterator object from the menu and once again iterates over the menu items.

```

package za.ac.cput.kabasob.designpatterns.behavioral.iterator;

import java.util.Iterator;

/**
 *
 * @author boniface
 */
public class MainDriver {

    public static void main(String[] args) {

        Item i1 = new Item("spaghetti", 7.50f);
        Item i2 = new Item("hamburger", 6.00f);
        Item i3 = new Item("chicken sandwich", 6.50f);

        Menu menu = new Menu();
        menu.addItem(i1);
        menu.addItem(i2);
        menu.addItem(i3);

        System.out.println("Displaying Menu:");
        Iterator<Item> iterator = menu.iterator();
        while (iterator.hasNext()) {
            Item item = iterator.next();
            System.out.println(item);
        }

        System.out.println("\nRemoving last item returned");
        iterator.remove();

        System.out.println("\nDisplaying Menu:");
    }
}

```

```

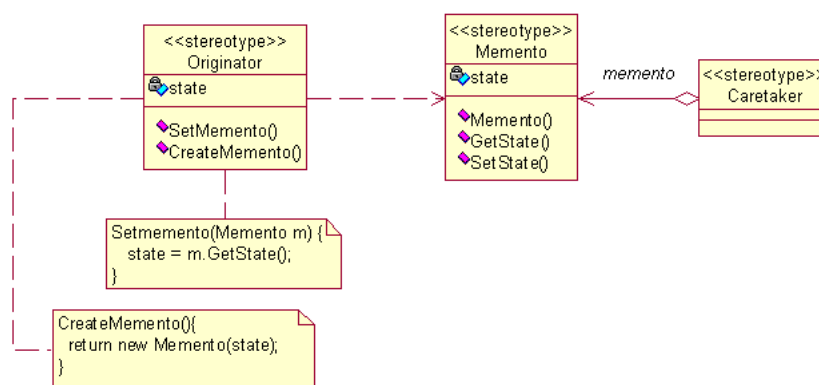
        iterator = menu.iterator();
        while (iterator.hasNext()) {
            Item item = iterator.next();
            System.out.println(item);
        }
    }
}

```

Record output in your log book

Note that since the menu utilizes a Java collection, we could have used an iterator obtained for the menu list rather than write our own iterator as an inner class.

7.7.4.10 Memento Pattern



The memento pattern is a behavioral design pattern. The memento pattern is used to store an object's state so that this state can be restored at a later point. The saved state data in the memento object is not accessible outside of the object to be saved and restored. This protects the integrity of the saved state data.

In this pattern, an Originator class represents the object whose state we would like to save. A Memento class represents an object to store the state of the Originator. The Memento class is typically a private inner class of the Originator. As a result, the Originator has access to the fields of the memento, but outside classes do not have access to these fields. This means that state information can be transferred between the Memento and the Originator within the Originator class, but outside classes do not have access to the state data stored in the Memento.

The memento pattern also utilizes a Caretaker class. This is the object that is responsible for storing and restoring the Originator's state via a Memento object. Since the Memento is a private inner class, the Memento class type is not visible to the Caretaker. As a result, the Memento object needs to be stored as an Object within the Caretaker.

Now, let's look at an example. The DietInfo class is our Originator class. We'd like to be able to save and restore its state. It contains 3 fields: a dieter name field, the day number of the diet, and the weight of the dieter on the specified day of the diet.

This class contains a private inner class called Memento. This is our Memento class that is used to save the state of DietInfo. Memento has 3 fields representing the dieter name, the day number, and the weight of the dieter.

Notice the `save()` method of `DietInfo`. This creates and returns a `Memento` object. This returned `Memento` object gets stored by the caretaker. Note that `DietInfo.Memento` is not visible, so the caretaker can't reference `DietInfo.Memento`. Instead, it stores the reference as an `Object`.

The `restore()` method of `DietInfo` is used to restore the state of the `DietInfo`. The caretaker passes in the `Memento` (as an `Object`). The `memento` is cast to a `Memento` object and then the `DietInfo` object's state is restored by copying over the values from the `memento`.

```
package za.ac.cput.kabasob.designpatterns.behavioral.memento;

/**
 *
 * @author boniface
 */
// originator - object whose state we want to save
public class DietInfo {

    String personName;
    int dayNumber;
    int weight;

    public DietInfo(String personName, int dayNumber, int weight) {
        this.personName = personName;
        this.dayNumber = dayNumber;
        this.weight = weight;
    }

    public String toString() {
        return "Name: " + personName + ", day number: " + dayNumber + ", weight: " + weight;
    }

    public void setDayNumberAndWeight(int dayNumber, int weight) {
        this.dayNumber = dayNumber;
        this.weight = weight;
    }

    public Memento save() {
        return new Memento(personName, dayNumber, weight);
    }

    public void restore(Object objMemento) {
        Memento memento = (Memento) objMemento;
        personName = memento.mementoPersonName;
        dayNumber = memento.mementoDayNumber;
        weight = memento.mementoWeight;
    }

    // memento - object that stores the saved state of the originator
    private class Memento {
        String mementoPersonName;
        int mementoDayNumber;
        int mementoWeight;

        public Memento(String personName, int dayNumber, int weight) {
            mementoPersonName = personName;
            mementoDayNumber = dayNumber;
            mementoWeight = weight;
        }
    }
}
```

`DietInfoCaretaker` is the caretaker class that is used to store the state (ie, the `memento`) of a `DietInfo` object (ie, the originator). The `memento` is stored as an object since `DietInfo.Memento` is not visible to the caretaker. This protects the integrity of the data stored in the `Memento` object. The caretaker's `saveState()` method saves the state of the `DietInfo` object. The caretaker's `restoreState()` method restores the state of the `DietInfo` object.

```
package za.ac.cput.kabasob.designpatterns.behavioral.memento;

/**
 *
 * @author boniface
 */
// caretaker - saves and restores a DietInfo object s state via a memento
```

```
// note that DietInfo.Memento isn't visible to the caretaker so we need to cast the memento ↔
to Object
public class DietInfoCaretaker {
    Object objMemento;

    public void saveState(DietInfo dietInfo) {
        objMemento = dietInfo.save();
    }

    public void restoreState(DietInfo dietInfo) {
        dietInfo.restore(objMemento);
    }
}
```

The MainDriver class demonstrates the memento pattern. It creates a caretaker and then a DietInfo object. The DietInfo object's state is changed and displayed. At one point, the caretaker saves the state of the DietInfo object. After this, the DietInfo object's state is further changed and displayed. After this, the caretaker restores the state of the DietInfo object. We verify this restoration by displaying the DietInfo object's state.

```
package za.ac.cput.kabasob.designpatterns.behavioral.memento;

/**
 *
 * @author boniface
 */
public class MainDriver {

    public static void main(String[] args) {

        // caretaker
        DietInfoCaretaker dietInfoCaretaker = new DietInfoCaretaker();

        // originator
        DietInfo dietInfo = new DietInfo("Fred", 1, 100);
        System.out.println(dietInfo);

        dietInfo.setDayNumberAndWeight(2, 99);
        System.out.println(dietInfo);

        System.out.println("Saving state.");
        dietInfoCaretaker.saveState(dietInfo);

        dietInfo.setDayNumberAndWeight(3, 98);
        System.out.println(dietInfo);

        dietInfo.setDayNumberAndWeight(4, 97);
        System.out.println(dietInfo);

        System.out.println("Restoring saved state.");
        dietInfoCaretaker.restoreState(dietInfo);
        System.out.println(dietInfo);
    }
}
```

Record output in your log book

Notice how the state changes, and how we are able to save and restore the state of the originator via the caretaker's reference to the memento.

7.7.4.11 Interpreter Pattern

This one has been left for you to do as an exercise.

7.8 Chapter Exercises

This exercise has two parts.

Part 1

The First part requires you to read and master 6 design patterns, two from each category. Write a small write up on the practical usage of the patterns you have chosen. Your solution should follow the pattern below.

1. Pattern Name and Classification
 - Creational
 - Structural
 - Behavioural
2. Intent Also Known As Motivation and Applicability
3. Structure and Participants
4. Collaborations
5. Consequences
6. Implementation
7. Sample Code
8. Known Uses
9. Related Patterns

Part 2

Re-Write the Design Patterns in Section, but use JUnit Tests in place of the Driver class used in the examples. Also write code to demonstrate the usage of the Interpreter Design Pattern.